

Bachelor Thesis

Extending Interoperability: Type-Safe Bindings and Polymorphic-Interop Between Julia and C++

by

Kai Erik Niermann

(2720905)

First supervisor: Atze Van der Ploeg
Daily supervisor: Atze Van der Ploeg
Second reader: Verano Merino

July 7, 2025

*Submitted in partial fulfillment of the requirements for
the VU degree of Bachelor of Science in Computer Science*

CONTENTS

Contents	2
Abstract	1
1 Introduction	1
1.1 Background and Motivation	1
1.2 Objectives	2
1.2.1 Research questions	2
1.3 Scope and Limitations	2
1.4 Structure of the paper	2
2 Foundations of Language Interoperability	2
2.1 FFI & ABI Fundamentals	2
2.1.1 Foreign Function Interface (FFI)	2
2.1.2 Application Binary Interface (ABI)	3
2.2 C-ABI & Wasm: A Comparative Lens	4
2.3 Tool Architecture Patterns	5
2.3.1 Similar languages	6
2.3.2 Dissimilar languages	7
2.4 Memory Management Strategies	8
2.4.1 Static strategies	8
2.4.2 Dynamic strategies	9
2.4.3 Hybrid strategies	10
2.5 Type-Representation Strategies	10
2.5.1 Static interpreting	11
2.5.2 Dynamic interpreting	11
3 Formalising Interoperability	11
3.1 Interop Tools as Partial Evaluators	11
3.1.1 Metaprogramming	11
3.1.2 Partial Evaluation	12
3.2 Records as Dependent Product Types	13
3.2.1 Different Models of Records	13
3.2.2 Abstract Types & Substitutability	17
3.2.3 Substitutability in Julia and C++	18
4 Case Study: Polymorphism in a multi-language Ray-Tracer	19
4.1 Project Context & Toolchain	19
4.1.1 Implementation approach	20
4.1.2 Choice of Binding Generators	20
4.1.3 Implementation setup	20
4.2 Design Challenges	20
4.2.1 Problem Statement	20
4.2.2 Metaprogramming in Jluna	22
4.2.3 Jluna's Internals	23
4.3 Implemented Extension	25
4.3.1 Representing Fields & Types	25
4.3.2 Semantics of Unboxing & Boxing	26
4.4 Evaluation	28
4.4.1 Benchmarking & Performance	28
4.4.2 Developer & User Experience	30
4.5 Limitations & Threats to Validity	30
4.5.1 Slow and varying generics support	30
4.5.2 Asymmetric metaprogramming features	31
4.5.3 Decentralized language evolution	31
4.5.4 Theoretical exploration vs practical needs	31
5 Conclusion	31

Extending Interoperability: Type-Safe Bindings and Polymorphic-Interop Between Julia and C++

Kai Erik Niermann

Vrije Universiteit Amsterdam
Amsterdam, The Netherlands
k.e.niermann@student.vu.nl

ABSTRACT

Programming language interoperability is a crucial aspect of software development. A common challenge arises when bridging languages with considerable semantic differences. With the increasing expressiveness of modern programming languages, interoperability systems should facilitate more expressive communication. In this paper, we survey the capabilities of existing FFI-based interoperability systems and assess the viability of more expressive interoperability by extending an existing tool to support bridging polymorphic types. Through our tool-design survey and practical case study, we demonstrate that creating and applying more expressive interoperability between languages exhibiting significant semantic gaps is feasible. However, it requires a nuanced understanding of the languages involved and presents key trade-offs that must be considered.

1 INTRODUCTION

1.1 Background and Motivation

Two or more pieces of software, such as languages following different design paradigms in their compilation approaches and linguistic properties, working together via some means of communication forms the basis of what most people would understand as software interoperability. Interoperability is no new concept; it naturally ties into how the software systems we interact with evolve. Abstraction has been central to innovations in programming languages, aligning them more intuitively with natural language and making them more efficient for programmers. As many new programming languages distanced themselves over time from the hardware on which they had to execute, we had to develop systems that translate these composite abstractions, which define most software, back into the basic codes that machines can apply to execute said abstractions. Two notable byproducts of this type of evolution are that

- (1) Performance characteristics in kind evolved with the abstractions we developed to design new programming languages.
- (2) Not all abstractions are created equal, so naturally, languages evolve in various directions, built on various execution models.

A tradeoff between performance and ease of use can lead to a natural, albeit conflicting, question of how to balance ease-of-use abstractions and the desired performance characteristics of granular, low-level hardware control. Introducing domain-specific or new language abstractions can additionally lead to questions about how language abstractions differ and in what ways these differences are more applicable to some domains than others. All these considerations -performance, ease of use, language-specific features, or, more abstractly, the unique and evolving qualities of

programming languages -should then naturally lead us to why we might want to bridge specific attributes to "get the best of both worlds."

A byproduct of the increasing number of programming languages, especially ones that improve upon older languages considerably in aspects such as memory safety and performance, has led to an increased effort to develop practical solutions for language interoperability both to create bridges to well-established legacy codebases and to gradually rewrite codebases to align with modern safety standards and performance needs. Accounting for this increase in programming languages, ahead-of-time (AOT) compiler designers generally adopted a modular architecture to provide an extensible language development framework that supports various languages. Extensibility in AOT compilers typically follows the pattern of a front end, which a language developer must implement, some language-agnostic intermediate representation (IR) middle-end, such as GIMPLE for the GNU Compiler Collection (GCC) or LLVM-IR for the clang compiler, during which compiler and language developers can write generic or custom optimization passes, and a back-end during which the optimized IR is transformed into machine code to target some instruction set architecture (ISA). Separation of concerns: having a front-end, middle-end, and back-end with AOT compilation provides flexibility, but it can still be limited because we are confined to the implementation framework of the compiler. Interpreters address the constraints of AOT compilers by providing a platform-agnostic runtime execution model that trades bare-metal performance for increased flexibility in language design, abstracting away the difficulties of low-level considerations. Just-in-time (JIT) compilers aim to leverage the runtime information gained from interpreted execution approaches and combine this with more traditional bare-metal compilation to achieve both runtime language dynamism and performance through paradigms such as Profile Guided Optimization (PGO), enabling the specialized recompilation of *hot*, frequently executed, code segments. Combining various execution models: interpreted, hardware compiled with IR and custom byte-code, and implementing runtime optimizations, usually means JIT compilers have a rather complex and bespoke pipeline leading to difficulties in their adaptation to a broader set of languages outside of the main set they where designed around. Increasing the modularity of JIT compilers or, more generally, the *virtual machines* (VMs) they are part of has been an active area of research and has had considerable innovations in both evolving VM ecosystems such as the Java VM (JVM) from Oracle with Java, Kotlin, and Scala in addition to providing frameworks to implement new languages such as the GraalVM [40] or Microsofts Common Language Runtime (CLR) [52] C#, F# and Visual Basic

.NET (VB.NET) with community projects for other languages like C++, JS, Python and so on.

Reconciling the distinct qualities of AOT-compiled languages, such as C++, with those of a modern JIT-compiled language, like Julia, in the context of interoperability is a primary motivator for this paper. Further considerations include not only the practical ramifications of bridging Julia and C++ but also the insights we can gain from attempting to understand a more unified picture of how to bridge this, in some ways, divergent evolution of different programming language features, especially in a contemporary context. We will consider two angles in this research. To provide a foundation for understanding the research’s more practical and applied elements, we will elaborate on the background that underpins the most common form of interoperability and how this takes shape in modern interoperability tooling, thereby contextualizing the later case study within the broader landscape of the tooling. We will also explore a grounded, more applied perspective by performing a practical case study through the application and extension of an interoperability tool called Jluna for C++ and Julia. As both Julia and C++ have an overlap in being used in scientific and high-performance-computing (HPC) applications, this case study will specifically be centered around using C++ to implement a performance-intensive segment of a ray-tracer implementation in Julia mirroring how one might use existing well-established C++ libraries to accelerate performance-critical segments of a larger application written in a more dynamic modern language such as Julia.

1.2 Objectives

Primarily, the objective of this work is to present a tool-based design and case study in applied contemporary language interoperability research with a focus on system design, practical implementation, and language semantics. Integral to understanding most concepts one learns throughout one’s academic and professional career is building a mental structure for what defines the shape and various aspects of a specific domain. Interoperability, being an expansive domain, can then be difficult both to build a larger structure of and understand how it ties into some of the broader aspects of programming languages. A central takeaway is that, despite the expansive nature of the language interoperability domain and the seemingly irreconcilable attributes of divergent language and compiler design paradigms, a structure can be understood and leveraged to extend tooling to fit modern programming language semantics better. Decomposing our objective into research, we explore the following.

1.2.1 Research questions.

- (1) What are the foundations of FFI-based tooling, and how can WebAssembly (WASM) innovate on C-ABI-based FFI approaches?
- (2) How do interoperability tools approach system design, memory management, and type representation challenges, and how do these approaches vary based on language and runtime similarity?
- (3) How can we discuss interoperability tools in the context of type theory, and how does this understanding inform the design of binding generators?

- (4) What are some of the ramifications of extending existing tooling to support polymorphism across AOT and JIT compiled languages?

1.3 Scope and Limitations

FFI-based contemporary interoperability tools and related background and theory will be the primary focus of the research. VM language implementation frameworks, such as the recent GraalVM, will be mentioned, though not thoroughly evaluated. Additionally, we will limit ourselves to the main tooling, which has been developed over the past 10 years or so and is primarily aimed at practical industry applications, as opposed to more niche use cases. We will also elaborate on some necessary background to understand the basic technology this interoperability tooling is built on, though, with a focus on its application in a modern context and evaluating it alongside some potential modern successors. We will also introduce theoretical formalisms, primarily as a means of presenting some of the practical implementation considerations through a theoretical lens rather than as a means of rigorous formal verification or proposing any new theorems. Situating the extension case study within the context of similar tooling and some more traditionally theoretical concepts is the predominant focus of this research.

1.4 Structure of the paper

We will begin by introducing the foundations of language interoperability, practical considerations, and some future-facing technologies, such as WASM, which could serve as a new basis for modern tooling to assess RQ (1). Evaluating RQ (2) will occur in three parts: first, examining some of the more general strategies and design patterns of interoperability tooling, we observe applied for languages with varying degrees of overlap in their design and implementation; second, focusing specifically on memory management strategies in the context of AOT, JIT, and hybrid AOT and JIT compiled interoperability systems. For the last part, regarding RQ (2), we will assess how interoperability tools understand the types they aim to bridge and the relevance of reflection in that process. Finally, RQ (3) and RQ (4) will be explored by applying some existing theory to the field of language interoperability and, via the extension case study, to evaluate the ramifications of encoding and implementing polymorphism between an AOT-compiled language, C++, and a JIT-compiled language, Julia.

2 FOUNDATIONS OF LANGUAGE INTEROPERABILITY

2.1 FFI & ABI Fundamentals

2.1.1 Foreign Function Interface (FFI). Motivated by a desire to utilize some code in a *guest language*, such as C, which does not exist in a *host language*, such as Python, for example, we have FFI libraries. FFI libraries, as the name might suggest, are typically libraries written for a host language that provides an interface, *i.e.*, some ‘user-friendly’ abstraction, to utilize functions from a guest language. Despite this standard division of *guest* and *host*, FFI libraries can often have segments written in both the guest and host language. However, a commonly shared and important attribute is that these libraries (despite the bidirectional nature of how data

might move) don't necessarily have bidirectional interfaces; their interfaces are often developed around the paradigm of using the guest code from the host code. Central to the development of FFI libraries is their reliance on a stable underlying *Application Binary Interface* (ABI). As communication between languages at runtime is fundamentally tied to their compilation systems, and thus the underlying ISAs we are compiling to, FFI libraries are typically built around some standard ABI. C compilers across platforms have adopted a stable and consistent specification for emitting code at the binary level. This has resulted in the most common type of FFI library being based on C-ABI conventions. An implication of the choice to enable interoperability primarily through the stability afforded by the C-ABI and the low-level simplicity of C's semantics has led to many mainstream languages such as Julia [71], Rust [74] or Python [5] having basic FFI-library implementations as a part of their standard libraries or early language histories.

We can construct a minimal example of what it might look like to interact with an FFI library as a programmer. We can write a small 'greet' function in C in a file called 'mylib.c', which we compile into a shared object (.so) format. In Julia, Rust, and Python, we can then broadly describe the common steps for loading and dealing with the library 'mylib.so' as follows

- (1) *Loading* - Load the .so library into your host language code.
- (2) *Preperation* Prepare your host language variables for the guest-language foreign function.
- (3) *Usage* - Declare and/or directly access the desired foreign function through the provided FFI.
- (4) *Handling* - Handle the return value and possible exceptions.

```

1 // mylib.c
2 #include <stdio.h>
3
4 void greet(const char* name) {
5     printf("Hello, %s!\n", name);
6 }

```

Listing 1: FFI Example in C

```

1 ccall(:greet, "mylib"), Cvoid, (Cstring,), "World")

```

Listing 2: FFI Example in Julia using the standard ccall function

```

1 #[link(name = "mylib")]
2 extern "C" { fn greet(name: *const c_char); }
3
4 fn main() {
5     let name = "World";
6     let c_name = CString::new(name).expect("CString::new failed");
7     unsafe {
8         greet(c_name.as_ptr());
9     }
10 }

```

Listing 3: FFI Example in Rust using extern and the link attribute for the library

Although generally describing any piece of software that provides an abstraction for handling foreign code, FFI libraries are most commonly associated with specifically C-FFI libraries, which is to say, interop tools in which C is the guest language. For this paper,

```

1 mylib = ctypes.CDLL('./libmylib.so')
2
3 mylib.greet.argtypes = [ctypes.c_char_p]
4 mylib.greet.restype = None
5
6 name = b"World"
7 mylib.greet(name)

```

Listing 4: FFI Example in Python using the cffi library

we will consider FFI libraries in the broadest possible sense, so tools with arbitrary source and guest languages, albeit still generally built upon the C-ABI as the intermediary. In the examples of different languages interacting with C, there are some important observations we can make, especially regarding the nature of the *preparation step*. Conformance to the C-ABI means that any pieces of data that pass into this binary code compiled for said specifications must, in turn, also conform to this specification. Coercing data to fit the C-ABI conventions leads to a kind of linguistic bottleneck where types that have language-specific usage paradigms (*i.e.*, structs with specific unique offsets or metadata) have to be deconstructed into a valid format to transfer across the language boundary and then uniquely reconstructed within our guest language if they want to mirror idiomatic usage or retain certain information. Deconstruction of types we saw during the preparation of translating language-specific string formats into C-strings and reconstruction we will encounter later.

Function signatures are another important aspect of this reliance on the C-ABI. Similarly, with parameters, the signatures themselves cannot natively exist, by which we mean without valid abstractions if they don't have some low-level analog. Generics are a prominent example of this because generics tend to rely on language-specific approaches for mangling overloaded function names with type-specific information to create unique signatures; even if two languages have generics and maybe even similar approaches for instantiation, a limiting factor will first and foremost always be the fact that the C-ABI, by itself, does not have any specification for mangling. Hence, something as we see in listing 5 cannot exist without deviating from a pure C-ABI-based FFI-library.

```

1 extern "C++" {
2     fn add<T>(a: T, b: T) -> T;
3 }

```

Listing 5: Pseudocode example of extern C++ generics in Rust

While providing a stable common ground to build FFI libraries on, the C-ABI, regardless of how much of the preparation one can abstract with FFI libraries, forms a very minimal and often limited basis for the development of interoperability tooling. Marshalling modern language features, especially those that aid in code reuse, requires various, often language-specific, strategies to evolve these libraries from their basic set of bridgable information.

2.1.2 Application Binary Interface (ABI). An important question that arises, especially as a language grows, is how changes to its binary code generation process deviate from older ABI specifications and hence impact compatibility with source code compiled

with implementations based on said specifications. To mitigate communication issues between binaries compiled following different ABI conventions, compilers tend to adopt specific conventions for a particular set of compiler versions, generally called *partial ABI stability* as we do have a standard convention for a set of compiler versions though no guarantees that this will continue indefinitely. *Full ABI stability* or *ABI stability* is, in contrast, colloquially understood to mean that from a particular compiler version onwards, all application binaries will adhere to a set of standardized conventions code generation conventions. Swift’s ABI is a recent example of reaching stability with it first introducing runtime stability in 2019 [63], enabling compatibility between binaries at runtime as long as they guested the stable ABI (where compiled with a swiftc version ≥ 5.0), and compile-time stability enabling developers to use precompiled frameworks within their source code for compiler versions past ≥ 5.1 [58].

The persistence of the C-ABI as the de facto standard for implementing FFI libraries can, in large part, be attributed to its stability and the legacy nature of C as a language. Despite the aforementioned limitations for more complex tasks, the C-ABI’s somewhat constrained and minimal specification, given its existing popularity, served as an effective basis for bridging languages whose compilers were capable of implementing its conventions. Utilizing often language-specific features such as compile-time code generation (i.e., macros in Rust, or templates in C++) as we see with CXX for Rust $\leftrightarrow$ C++ [37] or leveraging languages runtime models to introduce more dynamic behavior such as enabling runtime recompilation as we can the work of JavaCPP [6] has allowed developers to expand considerably on the more limiting core attributes of the C-ABI. Performance overhead nonetheless remains a fundamental aspect of working with C-ABI-based FFI libraries, as most innovations still generally add more layers of indirection that a variable or object must pass through. Effective memory management, specifically limiting unnecessary copies and performance-intensive memory manipulations, has been a crucial mitigation strategy for FFI performance overhead, often coming in the form of wrapper types such as in the case of the Julia $\leftrightarrow$ Python library JuliaCall, which innovates on an older library, PyCall, by never copying mutable objects across the language boundary by default [64]. Integrating more complex features into C-ABI FFI libraries to bridge two languages tends to require a clear focus on balancing the level of runtime indirection and object manipulation, as well as understanding how to most effectively encode and transfer objects across a primarily C-based intermediary language pipeline.

2.2 C-ABI & Wasm: A Comparative Lens

Painting a more practical picture of what it means for an FFI library to be C-ABI based, we can introduce two languages: our guest language L_g and our host language L_h . Central to most FFI implementations based around the C-ABI are usually two pieces of code: the library core, which generally consists of a dynamic library compiled to a platform-specific C-ABI target, and the guest API, which provides methods that allow accessing the dynamic library, usually built on more fundamental features provided by L_g ’s runtime or compilation model. Conformance to the C-ABI is what forms the basis of how the guest API is implemented. Resulting from this choice

to enable our language to be compatible with C-ABI compliant binaries, such as our dynamic library, is the fundamental implication that we need to coerce our types into C-compatible formats as a contractual obligation in the guest API. Coercion of types, in this instance, into a C-compatible format is what is generally known as boxing. Boxing, in the more general sense, can be understood as the concept of abstracting away the underlying properties of types to create a generic “box” that contains them. Programming language designers commonly use boxing in instances where the details lost in abstraction are not necessary or wanted by the programmer in a particular instance, a basic example being the `Number` class in Java. Boxing in the case of interoperability requires considerable nuance and often a considerable effort in how the Box is constructed, as the datatypes provided by C only provide the most fundamental constructs, such as primitives, structs, etc.—composing these to encode accurate and sufficient information to utilize the type properly when unboxing is critical. Reconstructing the type in a way we can effectively understand it in the target language is what is generally understood by *unboxing*. Unboxing has been implemented using a large variety of approaches, as generally, the goal is to be able to use a type idiomatically within the context of a language. Most design paradigms have centered on balancing performance with an idiomatic understanding of a type.

Finding the balance between performance and idiomatic representations of types comes down to how effectively the low-level encoded representation of a type from the guest language can be understood and applied in the target language and vice versa. Naturally, this leads to languages whose design philosophies are closely aligned with that of C, i.e., low-level, performant, systems-based languages such as Rust, C++, and Zig, which have a more straightforward path for exchanging similar types, as their memory layout is similarly minimal. Conversely, languages that deviate in both execution models (JIT instead of AOT) and design paradigms (flexibility over performance) tend to be more complex when working with types that exhibit boxing, require metadata, and have a more complex memory layout within the language. Nonetheless, we have seen both languages focused on dynamism exist that borrow from AOT languages such as Julia with its integration of LLVM-IR into its JIT compilation pipeline and strong tie-ins with C being both written in C and providing an extensive C-FFI library, and languages focused on performance, such as a Rust and C++ integrate features which enable more compile-time and runtime dynamism such as Rusts macros system or C++ compile-time code generation via template metaprogramming. Despite these innovations in programming languages and the corresponding adaptations made to C-ABI-based FFI libraries to bridge modern features, the increasing complexities of language semantics and execution models pose a valid question about the extent to which the C-ABI should continue to be utilized in the development of modern interoperability tooling.

In contrast to the C-ABI’s predominantly hardware-based nature, a commonality observed among various VM-based runtime ecosystems is that as they mature, they tend to implement a platform-independent intermediate code, known as bytecode, to provide the necessary extensibility for incorporating new languages. Bytecode is typically executed at runtime via an interpreter as a part of a VM or compiled via an implementation of JIT compilation. Some

nuance exists in the terminology, as there are unique instances, such as with Julia, which, while JIT compiled, does not utilize bytecode; instead, it opts for the more traditional approach of using just LLVM-IR, which it compiles to native code [72]. Two of the most widely targeted instances of bytecode are JVM-bytecode via the GraalVM [38] and *WebAssembly* (Wasm). First announced in 2015 and having its design ratified in 2017 [84], Wasm is a bytecode format designed to be executed within web browsers using stack-based VMs. Despite the initial runtime target of stack-based VMs, there are now various runtimes, such as AOT compilers that compile Wasm to native code, like WasmEdge [83], or JIT compilation implementations like GraalWasm [39]. Some of the most common and important compiler toolchains for Wasm are listed in Table 1.

Emscripten was an early toolchain that enabled compiling native C and C++ code to Wasm, providing a virtualized POSIX-like environment for programmers. The provided POSIX-like environment means that, at the API level, integrating C or C++ code with Emscripten allows native features, such as file system access, to utilize interfaces similar to those found in a standard POSIX/Linux API, requiring minimal changes to existing native code [29]. The use of a virtual POSIX-like layer is primarily motivated by the expectation that Wasm binaries will be executed within JavaScript or Wasm-supported VMs, such as browsers or Node.js. However, the close coupling between the compiled .wasm code, its runtime, and the ABI specification—as seen with Emscripten—can become problematic for server-side¹ Wasm runtimes. This is because Emscripten’s rapid and flexible development of POSIX emulation, while beneficial for browser compatibility, can hinder the stability and broader adoption of its binary specifications [3], [4]. In other words, the flexibility that enables Emscripten to quickly improve POSIX emulation also makes it less suitable as a stable foundation for server-side or long-term ABI standards.

Aiming to address the issues of ABI stability and the broader adoption of Wasm into server-side runtime models, the WASI subgroup has developed and is continuing to work on a set of standards under the WASI name [82]. A significant focus by WASI and the Bytecode Alliance in simplifying interoperability and reaching ABI stability has been the introduction of the Canonical ABI. Similarly to the C-ABI, the Canonical ABI aims to standardize how data moves across language boundaries. To standardize interop with the Canonical ABI, the approach taken is to use an *Interface Definition Language* (IDL) called *Wasm Interface Type* (WIT). WIT is used to define specifically what features a library offers; this is segmented into three levels of abstraction: an *Interface*, which represents types and functions; a *World* representing a group of interfaces and a *Package* representing a set of WIT files also known as *Components*. Binary compatibility is ensured through an interface-centric design approach. Language-specific tools generate bindings from your WIT code, creating a scaffold for your implementation. When compiled with a Canonical ABI-compliant toolchain, this ensures compatibility with any other Wasm binary compiled the same way, regardless of the source language [13].

Interoperability facilitated through the Component Model and its underlying Canonical ABI has a few benefits over C-ABI-based

approaches. Since compatibility between binaries is ensured by the nature of having an ABI-compliant implementation scaffold, no boxing or unboxing between types is required. Canonical-ABI-compliant toolchains, in essence, eliminate the need for boxing and unboxing, as ABI-compliant interface encodings are an inherent property of the generated bindings. The obvious drawback of this approach is that existing libraries that utilize language-specific features not representable via WIT do require some compliant WIT-encoded representation or risk incompatibilities, though, unlike the C-ABI which on the API level is based on C’s type model, WIT models a much wider range of modern language features as a part of its core specification then could be trivially encoded and compiled following the C-ABIs standards. WIT is designed from the ground up as an IDL, specifically for modeling interfaces. We can then generate ABI-compliant bindings using the modeled interfaces. This makes WIT a strong candidate to replace C’s type system as the de facto standard for interface specification in traditional FFI libraries.

2.3 Tool Architecture Patterns

There are three important attributes shared among the most popular interoperability tools: *Mediation*, *Validation*, and *Generation*. Mediation is what tends to exist at the core of most implementations; it refers to the approach that is used to reconcile semantic differences across language boundaries. Earlier, in Section 2.1, we discussed the C-ABI as a popular choice to serve as the common ground in the mediation process—a more recent alternative to ABI compatibility, as the common language has shifted to API compatibility. API compatibility is generally more popular in managed languages, *i.e.*, JIT compiled or interpreted, which integrate APIs into their runtimes which allow extending a language with native modules that comply with the API’s high-level conventions. The primary difference between interoperability at the API and ABI levels is how compatibility is ensured. Compatibility at the API or source-code level is enforced through high-level language constructs, which define a contract for how to interact with a natively compiled binary. At the ABI level, compatibility is enforced by languages following a standard platform-specific ABI specification. Though to clarify, ABI and API compatibility are not mutually exclusive, when an interpreted or JIT-compiled language calls API-compliant native code, ABI compatibility must still be ensured, the idea being that if a piece of native code is compliant with a language API, then safe, or ABI compliant, interaction should be guaranteed through the implemented higher-level constraints.

Generation and Validation are typically the guiding factors in implementing safe API compliance. In practice, API compliance serves as the bridge that connects code in our guest language to idiomatic code in our host language. Code that is API-compliant should typically behave according to the host’s language semantics, which means it should be written in an idiomatic fashion. Marshalling code typically occurs when translating guest language interfaces into ones that adopt host language semantics. The tools that interpret and generate compliant or valid interfaces, also known as bindings, to enable a safe exchange through some mediation layer, are aptly named *binding generators*. We can classify binding generators into two informal subtypes: ones aimed at creating API-compliant

¹Server-sided runtimes, in this case, are not to be confused with the traditional server-client model of code execution. Wasm code executing on the server side generally means “outside of the browser or other sandboxed runtime.”

Compiler/Toolchain	Description
Emscripten	Emscripten is a toolchain for compiling C/C++ code to Wasm. It introduced a POSIX-like API for Wasm, though this ABI has not been formally stabilized. [86]
Cheerp	Cheerp supports compilation to both WebAssembly and JavaScript, facilitating seamless interoperability with web environments. It also integrates with ts2cpp, a utility that generates C++ interfaces from TypeScript types [28], [61].
WASI	The <i>Wasm System Interface</i> (WASI) subgroup [82] aims to define a stable Wasm-ABI for native interfaces. In conjunction with the standards defined by the WASI subgroup, the Bytecode Alliance, in conjunction with various contributors, has been working on various tooling [10]. • wasienv: A higher-level toolchain supporting languages like Swift and C++ [4]. • wasi-sdk: A lower-level compiler toolkit requiring language-specific frontends [34]. • wasmtime: A standalone Wasm runtime implementing WASI [12]. • wasix: A compatibility layer that extends WASI with POSIX-like features [68]. • wasm32-wasi: A Rust-targeting WASI frontend integrated with the cargo build system [11]. WASI currently offers one of the most comprehensive and actively documented ABI models for programming Wasm-based systems.
GoJS / TinyGo	TinyGo is a lightweight compiler designed for resource-constrained environments, capable of compiling Go code to Wasm. Its documentation and ecosystem are still evolving. [7]

Table 1: Overview of Major WebAssembly Compiler Toolchains

bindings and ones aimed at creating ABI-compliant bindings. Generating API-compliant bindings means the implementation tends to focus on translating guest language interfaces into something that the host language’s API can interpret.

On the other hand, generating ABI-compliant bindings primarily revolves around ensuring that the shared generated interfaces in both languages, once compiled, have binary-level compatibility. What differentiates ABI-compliant bindings from API-compliant ones is the specification of the mediation strategy they are built around. API-compliant bindings utilize the native API specification from the guest language to create host language-native API wrappers around guest language interfaces and data. An intermediary language typically forms the de facto API specification for ABI bindings. The byproduct of relying on an intermediate language, assuming neither the guest nor the host is also the intermediary, is that both languages must have compliant interfaces to facilitate exchange. In practice, the difference in tooling built around API vs ABI-compliant binding generation often comes down to whether a language already has an API for integrating native code or not.

Validation, ensuring compliance with the generated bindings, can be separated into build-time and run-time verification. At build time, verification is generally implemented through either translations or intermediate wrappers, which are based on either the guest language’s native API or the API of the underlying ABI we are targeting, e.g., C’s interface and type semantics for the C-ABI. Build-time verification is primarily focused on establishing type-safe mediation mechanisms through conformance to the respective specifications. Run-time verification refers to approaches for validating messages across language boundaries, primarily to address language safety features (such as memory management and type checking) that are lost when using features from a guest language interface. To evaluate RQ (2) concerning the impacts of different language dynamics, such as memory management approaches, on design patterns in interoperability tooling, we will examine both tools for languages with similar paradigms and those aimed at more distinct domains.

Syntactically speaking, aside from very esoteric or functional languages (e.g., brainfuck, Haskell), programming languages roughly follow similar patterns in their design, *i.e.*, loops, functions, data structures, and variables. Semantically, “similarity” is a more nuanced point to consider. We can narrow down the semantic similarity of languages if we understand it as the behaviors a language exposes to move data throughout its execution. We can break this down into languages that focus on imperative procedures, exposing granular tools for manipulating memory, typically described as low-level or system languages, as they commonly exist close to bare-metal machine code. Languages that are based primarily on declarative, or simply high-level constructs, which abstract common instances of imperative control flow, then tend to form the other semantic group. The strictness of typing is another point that can be used to delineate programming language semantics, but for our purposes, we will use the perspective of memory management during control flow as it can provide some better insights in the context of evaluating RQ (2). Nowadays, we often see, especially for more mature languages, various back-end targets (JIT compilation, machine code, etc.), which in turn lead to many languages having features that deviate from their initial semantic focus. Nonetheless, the core design focus still tends to define the most common ways a language is used.

2.3.1 Similar languages. To evaluate tools aimed at similar languages, we will use two pairs of languages, (C++, Rust) and (Julia, Python). Tools that bridge Rust and C++ most commonly position C++ as the guest language, with Rust being the host language. Interoperability tooling developed around languages with low-level dynamics tends to exist primarily to reuse or rewrite libraries from an older low-level language with a newer one that has similarly good performance capabilities but innovates in areas such as type and memory safety. A relevant example of this is the various tools developed to integrate and rewrite C++ libraries within the Rust ecosystem. Driving the move towards Rust are both internal features of the language and related external factors, government measures [57] and industry investment [66]. Since minimizing overhead

is a core tenant of both Rust and C++, neither of their compilers comes with built-in API compliance. Due to the established nature of legacy C codebases, compilers for Rust and C have ways to compile C interfaces and produce compatible binary code via the C ABI standard. Rust-centric C++ interop tools, such as `autocxx` [69], `cxx` [77], or `zngur` [47], all fundamentally rely on Rust’s and C++’s feature of declaring and compiling C-compatible interfaces as the mediation strategy. The implementation for generation and validation is mainly dependent on how different people answer the question of how to parse and apply header files. `Bindgen` [76] uses `libclang` to consume C and C++ header files, which yields an Abstract Syntax Tree (AST) representation of the provided interfaces [16]. Using the AST representation of the interface, `bindgen` then generates Rust source code utilizing its C-ideological. `Autocxx` builds on top of a fork of `bindgen` and `cxx`, using the parsed headers from `bindgen` to generate the custom C++-IDL used in `cxx`. From the generated C++-IDL, it relies on `cxx` to generate the underlying C-IDL representing the C-ABI-compatible glue code. The primary benefit of integrating the generation and parsing from `bindgen` with the C++-IDL from `cxx` is that it combines automated header parsing with automated IDL generation, essentially removing all burden on the programmer to write any glue code [25].

The IDL provided by `cxx` and its corresponding transpiling mechanism from the C++-IDL to the C-IDL also provides a more idiomatic and safe way to work with declaring and interacting with C++ interfaces in Rust than older, more direct generation mechanisms [70]. `Zngur` differentiates itself from `cxx` and, by extension, `autocxx` by not embedding its C++-IDL within Rust; instead, it chooses to have it specified externally in a `.zng` file [48]. The approach `Zngur` takes parallels the interface-driven design we discussed earlier with `WIT`. While most popular interop tooling focuses more prominently on enabling cross-language dynamics through interfaces, emulating the traditional style of importing logically separated components, we also see tooling that aims not just to have embedded cross-language IDLs but to embed foreign code itself. An instance of a language-embedding tool for Rust and C++ is the `cpp crate` [33]. Despite still relying on the common C-IDL and using macro expansion to generate the compliant interface definition automatically, it takes a unique approach by allowing inline C++ code within our Rust source. At build time, the C++ crate utilizes macro expansion and parsing of the inline C++ code to generate a static C++ library comprising the inline code, which is then linked to the final Rust binary.

The use of compile-time code parsing and generation to implement the underlying binding generation, or even generating entire static libraries from foreign inline code, is a typical pattern among these tools, motivated in large part by the goal of creating zero-cost abstractions, as both languages are generally applied in performance-intensive domains. Having inline foreign source code draws some parallels to the approaches to interoperability through interpreter composition, as seen in the work of Barrett et al. [8]. A key distinction is how these approaches optimize crossing the language boundary. Barrett employs a technique called meta-tracing; tracing, by itself, refers to the idea of using information at runtime to evaluate hot code, optimize it (e.g., through specialization), and compile it to machine code. Meta-tracing focuses on tracing the

execution of an interpreter rather than the interpreter itself. A meta-tracing JIT compiler is then a JIT compiler that, unlike taking in the source code of a particular language, takes as input the syntax of an interpreter. The interpreter, in turn, executes a given source code and, while doing so, has its execution traced and optimized by the underlying JIT compiler. A benefit of using a meta-tracing JIT compiler, in this case, is that, unlike the C++ crates language composition, we are able to use runtime information to optimize objects crossing the language boundary. Zero-cost abstractions through compile-time code generation are an effective optimization technique, but their optimization capacity is limited as long as no runtime information is used.

Tooling for Julia and Python expands considerably in a more naturalistic composition of languages. For brevity, we will focus on tools where Julia is the host language and Python is the guest language, although various design patterns are naturally mirrored. To enable interoperability, both Julia and Python fundamentally rely on native language APIs. Julia, being executed by a JIT compiler written in C, naturally has a low-level C representation of objects [73]. Similarly, Python, running via the CPython VM, has a C API, which allows extending the language with low-level modules that conform to its API specification. Once again, we, therefore, see C forming the common ground here, with a primary difference between (C++, Rust) tooling being the application of language-specific native APIs to wrap objects, as opposed to transpiling towards a compatible intermediate C-IDL. As Python is a language built around dynamism, its object representations generally have sufficient metadata for easy parsing via Julia-sided code. This allows for wrapping the object in a mirrored interface that holds onto the underlying data, thereby avoiding the need for any copies. Both `JuliaCall` [64] and the earlier `PyCall` [45] follow the pattern of using low-level object parsing to create interfaces that can attach to Python’s C API object representation. To call back into Python code, the JIT compiler implicitly uses Julia’s `ccall` mechanism, which in turn interacts with Python’s C-API, in essence replicating any other C-based module extension from the perspective of Python’s VM.

2.3.2 Dissimilar languages. To assess the design patterns for tooling that bridges dissimilar languages, we will examine tooling that bridges Julia and C++. As discussed in Section 2.3.1, Julia provides a C API for low-level language extensions. Similarly, in the case of bridging Python to Julia, Julia’s C API is used as the target for wrapping code. A common inspiration we can note among modern tools that bridge C++ to a higher-level language, such as Julia, is the case of `CxxWrap.jl` [26] or Python in the case of `pybind11` [43] is the `Boost.Python` library [2]. The Boost C++ ecosystem is a set of libraries well known for their application of C++ templates and macros, essentially enabling compile-time code generation. Boost.Python, following the Boost paradigm, uses compile-time code generation to create the compatible Python C-API shims², or adapter functions, to integrate with C++ code. `CxxWrap.jl` follows a similar principle. On the user-facing end, it provides a set of macros and functions we can use to register C++ classes and functions. During compilation, the backend macros then translate

²The most common type of shim in interoperability are *thunks*, which adapt arguments to fit a particular calling convention and then trampoline to the actual function.

the interface declarations into a dynamic library, which exposes the registered functionality through a combination of Julia’s C-API and custom-generated shims.

Whereas CxxWrap.jl is focused on the more traditional guest-host language relationship, Cxx.jl [46] aims to achieve inline C++ code execution within Julia source code. The approach to inline foreign code execution in Cxx.jl differs from the cpp crate in that we don’t separate the languages at build time and then execute them through an intermediary. Instead, Cxx.jl executes the cpp code fundamentally through Julia’s `llvmcalls` and the application of *staged functions*. Staged functions, which are declared using the `@generated` macro in Julia, are functions that implement the JIT analog of overload resolution through compile-time specialization. Before generating machine code, Julia’s JIT compiler evaluates the types of the staged function’s arguments and then conditionally produces the machine code based on these types. Cxx.jl uses staged functions to select the correct C++ function to call, in essence specializing the C++ function using Julia’s JIT compiler. Once the inline C++ source has associated types, Cxx.jl uses Clang’s frontend to generate the AST and, from this, emit the LLVM-IR. Using the LLVM-IR Cxx.jl then relegates the actual execution of the emitted IR back to Julia’s built-in `llvmcalls` mechanism. Contrasting the approach taken in Cxx.jl, we can see that it, to a degree, bypasses the need for specific optimizations, either via meta-tracing or zero-cost abstractions, by simply emulating the native compilation flow of the target language, *i.e.*, compile-time specialization, a clang AST generated, and injected LLVM-IR execution. We can compare Cxx.jl’s approach to that of the Python-C++ tool `cppyy` [49]. Both `cppyy` and Cxx.jl can execute inline C++ code efficiently by embedding different compilation models into the host languages runtimes. Cxx.jl implements a de facto C++ JIT compiler by embedding parts of Clang’s frontend to parse and semantically analyze C++ code and using Julia’s LLVM backend (via `llvmlibcall`) to compile and inline the resulting machine code into Julia methods. `Cppyy`, on the other hand, embeds the C++ interpreter `cling` [80] to similarly execute the inline source in a way that idiomatically aligns with the underlying runtime of the host language, *i.e.*, the CPython VM.

Jluna [22], while superficially similar to CxxWrap.jl, takes a different approach in how it works with the Julia C-API and templates to generate the necessary bindings. CxxWrap.jl, despite using macros, still relies heavily on dynamically allocated objects to generate the necessary glue code. Jluna, on the other hand, uses an almost exclusively static approach. Jluna utilizes the static information from the declared C++ classes and functions to conditionally compile or specialize meta-types, which contain the necessary mechanisms and constraints to bridge themselves across the language barrier utilizing a symmetric type representation. Conceptually, we can understand the idea of implementing the dynamic wrappers we see in CxxWrap.jl or similar libraries as zero-cost abstractions. A significant motivation behind the design of Jluna was to provide an improved interface for Julia’s C-API, enabling interoperability with C++ code. As Jluna is still built on Julia’s C-API, we still see the same limitations in performance as with other more traditional cross-language tooling, *i.e.*, the overhead of boxing and unboxing, in addition to a sort of cold-start problem due to the extensive application of code generation. One way Jluna improves upon older

tooling is through its heavy application of template metaprogramming and conditional code generation, which enables it to express specific cross-language dynamics in a considerably more declarative style. The declarative nature of Jluna’s design stems in large part from the fact that, unlike wrappers being designed as dynamic objects meant to contain a set of types, Jluna’s meta-types are more clearly defined as self-contained, static type containers acting under compile-time constraints. A significant benefit of this is the additional ahead-of-time type safety guarantees gained both for users when defining types and for developers when extending the library.

2.4 Memory Management Strategies

Two important considerations come into play when discussing memory management in cross-language systems: the lifetime of a variable and how we handle allocation, deallocation, and access for different types across the language boundary. The implementation of languages often ties these concepts closely together, as we generally want allocation and deallocation to correspond with the expected lifetime of a variable to avoid persisting unused memory. Statically typed languages often rely more on manual control of memory, while dynamically typed languages typically implement some form of runtime garbage collection mechanism. Over time, we have observed that various popular statically typed AOT-compiled languages have developed approaches to limit the application of low-level memory control due to its notorious risks and often problematic usage. Examples include the Resource Acquisition is Initialization (RAII) paradigm in various C++ compilers like MSVC [53], Automatic Reference Counting (ARC) in Objective-C and Swift via the clang extension [15], the Garbage Collector in Go, and the borrow checker in Rust, which either determines implicit lifetimes at compile time or uses explicit annotations from the programmer to determine when to deallocate an object.

When examining memory management dynamics for different languages, we can observe two broad categories: languages that manage memory primarily through static compile-time constructs and languages that employ runtime implementations of heap management, *i.e.*, garbage collectors. While theoretically, any statically typed AOT-compiled language could have a garbage collector, historically, there has been a preference for management abstractions that model low-level, performant control of memory while providing sufficient safety and ease of use. The question then becomes how to reconcile such systems, focusing on bridging systems with different memory management strategies and types with specific mutation properties and levels of indirection. To evaluate this question, we will examine the memory management strategies of interoperability systems within the context of bridging AOT languages, JIT languages, and AOT with JIT languages.

2.4.1 Static strategies. Central to our analysis of interoperability in the context of AOT-compiled languages will be tooling built around C++ and Rust, as it provides some unique insights into abstractly similar languages but with specific, considerable differences in how they handle memory management. C++ and Rust are both statically typed languages that are compiled ahead of time; they both aim to provide high performance through granular control, static typing,

and minimal overhead-inducing abstractions. However, they employ different memory management models. C++ has traditionally relied on memory management techniques such as RAI and reference counting, whereas Rust utilizes a borrow checker to enforce memory safety at compile time. For the sake of brevity, we will limit our discussion to how objects, which are instances of record types, are managed across the language boundary. Within C++, a record type is referred to as a class type, which encompasses both non-union and union types. The lifetime of a class type in C++ is tied to its storage duration or temporary nature, which is determined by the context in which it's created and how it's declared, *i.e.*, static, new [23]. In Rust, a record type is defined as a struct, and the borrow checker determines its lifetime. At the core of how the borrow checker works is the concept of an ownership model. Ownership is a compile-time construct in Rust that deterministically ensures each value has a single owner. When the owner goes out of scope, the value is automatically dropped, placing it in an invalid state. The most straightforward approach for reconciling the semantic differences in memory management between C++ and Rust is to more or less side-step the issue through the use of thunks and *opaque type* pointers.

Thunks are adapter functions, *i.e.*, shims, which adapt arguments to fit a particular calling convention and then trampoline to the actual function. Tools such as cxx, autocxx, and the newer Crubit [35] work extensively by generating C-ABI compatible thunks via extern "C" blocks. Cxx handles memory layouts of types primarily through opaque types, which we can understand as types that persist as a pointer to some memory from a guest language within a host language. Opaque types enable us to conceptually move a type around in our host language, with a core limitation that we have no actual usage of the type beyond callbacks. Cxx does have a extern "C++" block as a part of its C++-IDL, which allows the programmer to define the Rust-sided interface implementation that internally relies on thunks. Autocxx and Crubit evolved the idea of Rust-sided wrappers by automating their generation process. As already discussed in section 2.3.1, autocxx uses bindgen to parse headers and then generate the C++-IDL. Crubit aims to avoid using a macro-based IDL and instead directly generates the interfaces for the other language, *i.e.*, Rust from C++ and C++ from Rust, and the associated thunks.

Although cxx, autocxx, and Crubit generate host-language interface wrappers that internally use opaque pointers and thunks, they differ primarily in how they allocate, manage, and release the memory for the underlying guest-language objects. As these tools are primarily centered on Rust as the host language, we will focus on liveness from the Rust perspective. An important aspect of the safe usage of objects across language boundaries is that their underlying memory remains in a valid state until both languages no longer use the object. In theory, the most trivial way to avoid premature deletion of objects is by having their ownership exclusively managed within a single language. A language taking ownership of objects it uses in a cross-language setting poses specific challenges. Native Rust objects virtually all change ownership via *destructive moves*. A destructive move leaves the *moved-from* object in an inaccessible state. The underlying implementation of destructive moves is essentially a memcp operation. While the approach of destructive

moves works well in conjunction with the compile-time borrow-checker model, we cannot trivially apply it to Rust-sided memory, which holds C++ objects. The subset of C++ types that do preserve their semantics within the context of destructive moves, *i.e.*, memcp moves, are called *trivially relocatable* (TR) types. Relocatability, in essence, aims to replicate Rust's destructive moves. Traditional moves in C++ via move constructors occur *non-destructively*, meaning the moved-from state must have its destructor in a valid state [41]. Trivially relocatable types are movable and provide no user-defined destructors [14]. As of mid-2025, most C++ types which are TR can be identified by either being trivial, such as scalar types, and ones that are marked with the `trivial_abi` clang attribute [17], [25]³. Unlike cxx and autocxx, Crubit aims to represent the memory layout of TR C++ structs within Rust by mirroring the underlying layout [24]. Mirroring the C++ layout enables Crubit to construct TR C++ objects in Rust-allocated memory. Objects that are not trivially relocatable, which is to say objects that we are unable to move with a memcp and a destroy, are a more complicated problem to address. Boxing via the aforementioned opaque pointers and interface wrappers avoids this issue but comes with undesirable performance overhead. A typical example of non-TR objects is self-referential classes, as they retain internal pointers to old memory locations when destructively relocated. Self-referential classes are something that C++ can manage with custom move-constructors, but Rust cannot. Rust introduced the `Pin<T>` wrapper type to address the issue of types that are address-sensitive and require pinning in memory, such as self-referential types. *Pinning* a location in memory provides the guarantee that the pinned value will remain located in the same place in memory from its creation till the time it's dropped [75]. An early application of pinning to interoperability with C++ was with the MoveIt crate [85], which used `Pin` to construct C++ objects into immobile Rust memory, either on the stack or heap, with `Pin<Box<T>>`. The same idea of using `Pin` for non-TR memory is applied in Crubit's design. To avoid objects having their destructors called from the C++-sided end, Crubit typically generates cross-language thunks as methods that pass raw pointers, thus avoiding either scope-based or reference count-based destructor calls.

2.4.2 Dynamic strategies. What we saw in Section 2.3.1 is that for similar languages aimed at dynamism, their interoperability is typically enabled through a low-level API, which allows native code to communicate with higher-level constructs. How we represent objects and manage their lifetimes once again becomes a focal point when assessing memory management. Memory management and garbage collection algorithms are typically automatic runtime processes integrated into the runtime of a particular language. JIT-compiled languages, such as Julia or Java, often have their memory management models defined by allocation heuristics and collection strategies, including mark-and-sweep. The dynamics of when we run gc-collection is also influenced significantly by memory pressure on the system and the generation classification of objects, *i.e.*, how many collection phases an object has survived [19], [27].

³Depending on if the proposal described in P2786R11 [51] is integrated into C++26 we could also see the type trait `std::is_trivially_relocatable` be introduced to more easily detect trivially relocatable types.

Because garbage collection is managed automatically and flexibly by each language’s runtime, cross-language memory persistence is typically achieved by explicitly instructing the guest language’s garbage collector (GC) via its API to keep referenced objects alive for as long as they are needed across the language boundary. Due to the semantic gap that exists between the underlying memory layout of object models within dynamic languages, it’s rare to see attempts to hold guest-language objects directly within the host language. Wrapper types are usually host-allocated objects that hold a reference to the foreign object and interface via the language’s embedding API, such as the JNI for Java, Julia’s C-API, or Python’s C-API [67].

2.4.3 Hybrid strategies. Hybrid approaches to memory management are employed in tooling, which aims to bridge languages that have fundamental gaps in their memory semantics, *i.e.*, managed vs. unmanaged languages. It’s an aim among most interoperability tooling to represent types idiomatically and with low overhead. Representing objects from managed languages within unmanaged languages is considerably more non-trivial than the other way around. As such, it’s common to see hybrid tooling taking the shape of the dynamic language handling static types. Pybind11 is a good example of the more traditional established designs. Within pybind11, the problem of object management is solved largely implicitly by interfacing with the guest language through the embedding API. Objects are typically heap allocated by default and then wrapped in a `unique_ptr`. The C++-sided `unique_ptr` object is then wrapped in a python-sided managed object, which the GC can keep track of. Once the Python-side object is garbage collected, the default behavior is to destroy the underlying C++ object as well. Embedding APIs, as opposed to, say, regular binding generator tooling, are typically designed to idiomatically extend dynamic languages with low-level code with a heavy focus on exposing different runtime internals. The design paradigms of embedding APIs are, therefore, quite effective at handling the memory management of objects from guest languages, provided said guest languages can generate the necessary bindings to the embedding API.

A more recent approach mentioned when discussing tool architectures was the method of embedding the static language more directly within the dynamic runtime. A nice example of this is `cppyy`, which, as a reminder, embeds C++ within Python through the use of the `cling` interpreter. Abstractly speaking, this is not too dissimilar from `pybind11`; both `cppyy` and `pybind11` use Python’s embedding API to expose the foreign language’s API. What differentiates `cppyy` from `pybind11` is that it leverages the introspective properties of the `cling` interpreter to generate the necessary bindings at runtime. In a sense, `cppyy` replaces the manual work of declaring the C++-sided bindings with the automated introspective parsing that `Clang` already implements. Despite the architectural differences, `cppyy` and `pybind11` ultimately result in similar memory management properties for Python-managed objects that hold references to heap-allocated C++ objects. A critical takeaway from this design is that only objects initiated from the dynamic side and, in turn, owned by the embedding API can be implicitly managed. All other objects, *i.e.*, those which are not initialized from the dynamic side, have to be explicitly managed by the programmer.

Another case to consider is how dynamic types are typically represented within static languages. One approach taken by two popular tools, namely `pybind11`, and `Jluna`, is to embed the dynamic language’s runtime within the static language. In the case of embedding the runtime within the static language, both `pybind11` and `Jluna` have implicit proxy layers that handle boxing and unboxing. These proxy layers are essentially minimal replicas of how the dynamic language runtime interacts with the embedding API. They generally aim to roughly emulate the semantics of the dynamic language and, in kind, leverage the underlying runtime, *i.e.*, the embedded interpreter for memory management. Naturally, the embedded interpreters typically also execute inline dynamic source code, which, depending on the implementation, can capture any scope of proxied variables as they are tied to the runtime through the embedding API.

A final approach worth discussing is specifically the symmetric type representation of `Jluna`. `Jluna` distinguishes itself from other hybrid models by its ability to altogether avoid wrappers *for the programmer*. Within most interoperability tools, wrappers define types that represent the foreign type within the host language. Another interpretation we can have of a wrapper is that the host type, which is implicitly defined by the wrapper’s constraints, is the actual idiomatically pure type within the host language. Wrapper constraints encapsulate a set of either static or dynamic assertions that define the semantics of how we interact with the foreign type. The most important of these constraints for record wrappers is typically the field typings, as we need the marshaling mechanisms to have a valid input type in the host that can be mapped to the target. The “front-end” representation we get from wrappers is what is then usually intended to mirror an idiomatic type within the host language; however, an important aspect is that the actual back-end semantics of the type, more often than not, don’t conform to the host language’s pure idioms. Let’s take, for example, a C++ struct from `pybind11` or `cppyy` on the Python side. We obtain an object that has the same fields as the C++ struct and behaves, for the most part, like a Python object; however, the actual semantics of the type are driven by the host’s embedding API rather than the host language runtime itself. When we talk about a purely idiomatic type, we mean a type that is defined purely within host language semantics, *i.e.*, a regular host language type. Here, we have `Jluna`’s symmetric type representation come into play. Unlike being used to contain a foreign type persistently, `Jluna` utilizes the notion of a wrapper as a blueprint to construct a purely idiomatic type that mirrors the foreign type. Similarly to `pybind11`, we describe a type of our interoperability layer. However, instead of generating a wrapper that contains the foreign type and manages it, `Jluna` generates wrappers that act as specialized or type-specific translation proxies. These translation proxies are then used in conjunction with wrapper lambdas to create an invisible middle layer of interoperability that yields purely idiomatic types within both the host and target languages.

2.5 Type-Representation Strategies

When we discuss type representation strategies, we refer to how interoperability tooling parses the syntax of a type and derives a semantic understanding from it, allowing it to form an internal

representation. We have touched on some of the design considerations for how we parse types when discussing design strategies in section 3.1. A theme that might have already become apparent from previous mentions is that type parsing in the context of interoperability often naturally mirrors how native types are interpreted. Parallels between type parsing in interoperability tools and type interpretation in host languages offer insight into key design considerations.

Within traditional models of compilation, we generally create a semantic representation of a type by first delineating the source code into units commonly called tokens through a lexer and then, from this stream of tokens, create a structural or syntactic representation of our code. A common organization of the syntactic representation is a hierarchical AST. During this *front-end* analysis, we typically also perform semantic analysis, which includes enforcing type constraints on data. Type checking in the front end is also commonly referred to as “static” type checking. In the context of interoperability tooling, we are generally exclusively interested in the semantic representation of types as opposed to the entire language. Nonetheless, due to the overlapping aim of deriving a semantic-preserving manipulatable representation of a type, compiler-based parsing has considerable influence on the design of interoperability tooling.

One way to understand traditional compiler front-ends at a high level is that they create a view of the source code and then generate a representation of this view, which can be manipulated, such as through optimizations, in a semantically preserving way. In the case of dynamic languages, the idea of viewing code often comes in the form of runtime introspection, which was touched on briefly in section 3.1. *Reflection* builds on the idea of introspection by also enabling self-modification of code at runtime. A key distinction between *Reflection* and the transformations that occur in a compiler front-end is that *Reflection* is not necessarily semantic-preserving across the entire program. Introspection and *Reflection* are often significantly more variable in the schema they use to represent types. However, due to their increasing application and development, they have offered another idiom from which to derive type representations.

2.5.1 Static interpreting. Two common approaches that fall into the vein of the compiler, or statically-based, interpreting are either emulating compilers, *i.e.*, building a mini compiler front-end aimed at lexing and parsing specifically headers, or using a compiler’s AST parser and then transforming this into a domain-specific representation. We can see the former approach in tools like *cxx* and, by extension, *autocxx*, which use the *syn crate* [78] for lexing Rust tokens and then feeds this into a semi-custom parser that handles constructing an AST for the rust-sided type representation, which can be used for validation and code generation. A generally more challenging task is transforming a whole language AST into a domain-specific representation. As mentioned in Section 2.3.1, the *bindgen* tool uses the *libclang* library, which provides a small and stable API for some of Clang’s front-end internals to parse C/C++ headers. Using *libclang* comes with various benefits and drawbacks; a primary one, as described by the *Crubit* team, is that while *libclang* provides stability, it also limits flexibility. Due to the limited flexibility provided by *libclang*, amongst other reasons, we

have also seen instances that opted for a tighter integration with the clang front-end, such as *Crubit* or the Swift compiler, which both import features more directly from the clang front-end source code for parsing C++ headers [88]. Swift also appears to use *libclang* in its compiler, although it does not seem to be the primary tool for parsing C++ headers.

2.5.2 Dynamic interpreting. Dynamic type interpretation is most commonly found in languages that offer the best facilities for introspection and reflection. Here, we can see introspection and reflection being applied in various respects. Tools such as *Jluna* use the metaprogramming features provided by *Jluna* to implement a meta-type that represents a generic container for records, which is then used to construct the actual type representation of the mirrored foreign type with the metaprogramming tooling. We also commonly see introspection and reflection applied in interop between static and dynamic languages. Since it’s common to use low-level embedding APIs to interoperate with high-level languages like Python or Julia, there is a strong incentive to exploit the linguistic overlap between these APIs and the static guest language. This enables the creation of translation mechanisms that don’t depend on external tools (e.g., header parsers). As a result, developers often rely heavily on metaprogramming to manually specify types in a way that emulates dynamic introspection. These specifications are then transpiled or transformed into API-compliant code that can interface directly with the high-level language.

3 FORMALISING INTEROPERABILITY

Specific shared patterns emerge when we discuss the design of various interoperability tools. The source of these shared patterns, in large part, stems from specific attributes of the problem they aim to address. One of these attributes is that, similarly to compilers, interoperability tools typically take as an input some static source and then, once compiled, yield a program that enables dynamic behaviors constrained by the semantics of the input source. We can understand this as a form of partial computation, which provides a potential lens through which we can discuss interoperability tooling within the context of the existing, well-established framework of partial computation.

How types are represented within interoperability tooling becomes another important cornerstone of their design. Type representations, *i.e.*, their models, such as how generic wrappers are defined, generally define the constraints on how we bridge and interact with types across the language boundary. The constraints of type representations often become more prevalent as language-specific models have less and less overlap, which can be leveraged to define common ground. As such, there is value in not just understanding the type models of individual languages but also their relations within a shared theoretical framework. In the latter half of this section, we will discuss specifically how dependent type theory can provide a clear picture in motivating confident, practical design choices for the joint interoperability tool.

3.1 Interop Tools as Partial Evaluators

3.1.1 Metaprogramming. As the practical anchor for our discussion of partial evaluation we will use the concept of metaprogramming, specifically within C++. We’ve mentioned previously that

metaprogramming is used in various interoperability tools such as Jluna and Pybind11 to generate the necessary bindings. When viewed from the abstract perspective what's happening in tools that employ metaprogramming is that the interfaces which specify the types generally don't manipulate data in any sense, but rather they manipulate code, *i.e.*, generate instantiated bindings. Manipulating code is a key aspect of metaprogramming, which is generally defined as the ability to write programs that can manipulate other programs. An interesting perspective and previously explored perspective on metaprogramming is that its instantiation or specialization process is a form of partial evaluation [81].

3.1.2 Partial Evaluation. We can introduce the concept of partial computation, also known as partial evaluation (PE). PE, first introduced by Futamura [32], states that given a function f processing some known, or static, data k , and unknown data u , we can generate an intermediate program f_k which is the result of computing everything in f with k and then computing $f_k(u)$ we can get a final result. Visually, we can represent this in the diagram 1

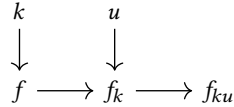


Figure 1: Partial Evaluation Process [32]

Definition 3.1 (Partial Evaluator). Using the more traditional notation from Futamura, we can define a partial evaluator p_e as a function that maps a program p and some collection of statically known information I_s to a program p' called the *residual* via specialization, with p' then mapping the dynamic information I_d (*i.e.*, runtime information) to a given output O .

$$p_e : \langle p, I_s \rangle \rightarrow p' : I_d \rightarrow O \quad (1)$$

Bringing this back to metaprogramming, an observation that was noted by Veldhuizen et al. [81] is that this act of specializing based on the static binding of types to templates is quite analogous to the process of partial evaluation. Something that further supports this idea is that templates allow non-type parameters, recursion, and partial specialization. Because template recursion is inherently self-referential, and a mix of non-type parameters plus partial specializations lets you state the base case, you can build arbitrary control-flow constructs—conditionals, loops, even finite-state machines—entirely at the template (*i.e.*, type) level, something which first discovered by [79]. Due to the somewhat accidental nature of the discovery template metaprogramming and the notoriously cumbersome syntax of C++ templates, the idea of applying this paradigm to an extreme extent has primarily been relegated to niche instances.

Definition 3.2 (First Futamura Projection). Some important observations we can take away from this similarity between template metaprogramming and partial evaluation is that firstly, we can view the process Jluna defines to specify and implement types as a form of partial evaluation. Specifically, we consider Jluna as a binding

generator generator, which means that Jluna represents a partial evaluator that takes the type specifications and the generic logic used for bidirectional transformations and defines a specialized binding generator, which is then compiled. Expressed in formal terms if we consider Jluna as a function G_{L_1, L_2}^1 which maps a set of type specifications T and a set of binding generator logic C to a binding generator C_T we can express this as follows.

$$G_{L_1, L_2}^1 : \langle T, C \rangle \rightarrow C_T \quad (2)$$

Definition 3.3 (Second Futamura Projection). If we view G_{L_1, L_2}^1 as an interpreter for a given type specification, then we can see this as representing the first Futamura projection. Abstractly speaking, Futamura projections express different stages of ‘meta’ computation; the first projection represents what was described above, a partial evaluator that specializes in a program based on some known information. The second projection then represents the notion of specializing the specialization for the interpreter. In the context of binding generators, this process involves specializing the binding generator for a specific set of languages. Formally, we could define this specialization as a function G^2 which maps a pair of languages $\langle L_1, L_2 \rangle$ to a binding generator G_{L_1, L_2}^1 as follows.

$$G^2 : \langle L_1, L_2 \rangle \rightarrow G_{L_1, L_2}^1 \quad (3)$$

In practice, implementing G^2 requires us to work at a higher level of abstraction. Instead of writing generic logic meant to apply to the set of types we wish to bridge, we would define the abstract attributes of a binding generator itself. This means that boxing and unboxing logic would be defined in a way that is independent of the specific types being bridged. Stated differently, instead of writing bridging functions for a set of types, we would use them for kinds or families of types. A basic example of this would be that instead of defining unboxing functions for, say, integers, booleans, floats, etc., we would define a generic unboxing function for the *primitive* type family, which would be specialized for each primitive type. Currently, we are not aware of any binding generator that implements a full-second Futamura projection. One instance that comes close is the Simplified Wrapper and Interface Generator (SWIG) framework [31]. While this tool technically accepts arbitrary target language specifications, its host languages are limited to C and C++. Furthermore, the tool does not specialize in a binding generator for the languages; instead, it directly generates the bindings based on the parsed C/C++ headers. Nonetheless, it does present an interesting case of a binding generator tool that leverages higher abstractions to define a more generic binding generator.

The complexity of implementing an accurate second Futamura projection is, in large part, a function of how we approach making the actual binding generator logic generic. Generating the correct methods for boxing and unboxing is relatively straightforward in theory, as it's more or less analogous to just specializing some generic code over a set of input types. The more complex part is how we can effectively generalize the logic of actually generating the value-level type conversions. One of the reasons why SWIG is specifically able to get at least part of the way there is that it leverages the fact that many languages can create binding generators that have similar embedding APIs, specifically for C. However, this

only works because the interoperability context is narrowly scoped. For a more principled and fully general second Futamura projection, the generator logic would need to abstract not just over surface syntax or API calls but over deeper interoperability semantics — including memory ownership, mutability, and type representation — across a much more diverse set of host and target languages. This significantly increases the design complexity and is part of why such systems remain largely theoretical.

An important thing to take away from the idea of thinking about binding generators as partial evaluators is that we end up with a more well-defined separation of concerns between different steps of the binding generation process, which can offer up a more effective foundation for implementing future generators in a more uniform and preformant manner by isolating the statically known specifications from the actual implementation logic and viewing the implementation itself as a specialized over the specification as opposed to being a direct implementation of interop logic. The relevance here is that it supports the fact that any extensions made to the type specification should be done in a way that does not compromise the existing logic of the binding generator, ensuring we maintain this quality of separation of concerns.

3.2 Records as Dependent Product Types

Records, in the most straightforward sense, are a collection of fields indexed by a set of labels corresponding to some value. The most basic form of indexing is typically implemented as an offset-based access to a contiguous block of memory. When discussing records within a formal framework, there are three prominent paths we can take. The first approach would be to view records through the object calculus (ζ -calculus) framework from Abadi et al. [1], which was motivated by a desire to formalize the semantics of object-oriented languages in the same way different λ -calculi act as the foundation for more procedural languages.

3.2.1 Different Models of Records.

Definition 3.4 (Records in the ζ -calculus). Within the ζ -calculus, records, or more broadly objects, are defined as a collection $i \in [1, n]$ of labels l_i and their corresponding values consisting of methods $\zeta(x_i)b_i$. In full, the notation is

$$o = [l_i \mapsto \zeta(x_i)b_i]_{i=1}^n \quad (4)$$

The method $\zeta(x_i)b_i$ is intended to represent the concept of a ‘self’ parameter, which each value of a record can access. Hence, a field access $o.l_i$ would be defined by the reduction.

$$o.l_i \rightarrow \zeta(x_i)b_i[o/x_i] \quad (5)$$

This is functionally no different from how most object-oriented languages with the concepts of a ‘self’ or ‘this’ parameter work.

The second way records are commonly modeled in an intuitive sense is through a structured ontological perspective. To illustrate this, let’s consider the following sentence and work our way through it.

a student writes a thesis.

We can create a structured ontology of this using the grammar defined by Montague [54], which would look like the following.

$$\begin{aligned} \text{‘a’ : Determiner} &= \lambda Q. \exists(P(x) \wedge Q(x)) \\ \text{‘(a) student’ : Noun} &= \lambda x. \text{STUDENT}(x) \\ \text{‘writes’ : Transverb} &= \lambda x. \exists y. \text{WRITES}(x, y) \\ \text{‘(a) thesis’ : Noun} &= \lambda y. \text{THESIS}(y) \end{aligned}$$

4

By combining these, we can express the sentence as follows.

$$\exists x(\text{STUDENT}(x) \wedge \exists y(\text{THESIS}(y) \wedge \text{WRITES}(x, y))) \quad (6)$$

The resulting proposition we get in 6 allows us to express the idea that there exists a student x who writes a thesis y . However, as a proposition, it’s limited to only expressing a predicate; from the proposition alone, we cannot derive any information about who the student is or what the thesis is. Nonetheless, this grammar conceptually captures the early idea of what constitutes a valid record initialization. In our example, the values of the record are the variables x and y , which are bound to the types `STUDENT` and `THESIS`, respectively. The predicate of the record is the relation `WRITES`, which we can understand as an invariant that is true for all records of this type.

The question that arises from this is how we can define the structural properties derived from Montague propositions while also retrieving the information about the values of our structure. Here, we can turn to *Type Theory with Records* (TTR). TTR is inspired by Martin L  f’s type theory and, in a sense, expands upon the idea of Montague’s grammar by introducing a structured object notation [20], [21]. Informally, TTR is a reflective representation of the Montague grammar. The notation for TTR is typically divided into type level and value level.

Definition 3.5 (TTR Records). A record type T is defined by a set of labels l_i and a corresponding set of types $T_i \in T$. The associated record object consists of a set of values d_i , which are assigned to the labels and correspond to the types specified for each label in the record type.

$$\underbrace{\begin{bmatrix} l_1 = d_1 : T_1 \\ l_2 = d_2 : T(d_1) \\ \vdots = \vdots : \vdots \\ l_n = d_n : T(d_n, \dots, d_{n-1}) \end{bmatrix}}_{\text{Record (Object)}} : \underbrace{\begin{bmatrix} l_1 : T_1 \\ l_2 : T_2(l_1) \\ \vdots : \vdots \\ l_n : T_n(l_1, \dots, l_{n-1}) \end{bmatrix}}_{\text{Record Type}}$$

Example 3.6 (TTR Record Instance). If we return to our example, we can express our proposition as a record using the TTR syntax as follows.

$$\begin{bmatrix} x = a : \text{Noun} \\ v_1 = p_1 : \text{STUDENT}(x) \\ y = b : \text{Noun} \\ v_2 = p_2 : \text{THESIS}(y) \\ v_3 = p_3 : \text{WRITES}(x, y) \end{bmatrix} \quad (7)$$

⁴A determiner is a word or affix that introduces a noun and expresses its reference in the context, such as “a,” “the,” “some,” or “every.”

Here, we retain the predicate nature expressed within Montague grammar through the dependence of types on earlier values. However, we also gain the ability to access values conforming to different types of predicates. A notable byproduct of this construction is that it provides a very explicit instance of the Curry-Howard correspondence. We can correspond the values and types of the record struct to proofs of the proposition 6.

Something mentioned a few times now is the idea of dependent types. TTR records are a direct application of dependent types, specifically to define the semantics of records. While interesting in their own right, situating TTR records within the context of dependent types provides a rich theoretical framework for reasoning about records and their semantics. To start, we can define what a dependent type is.

Definition 3.7 (Dependent Type). A dependent type is a type that is indexed by a set of values belonging to another type. This idea is represented as follows.

$$x : A \vdash B(x) : \text{type} \quad (8)$$

This means that for some value of type A , we can define a type $B(x)$.

A dependent type by itself only describes the family of types indexed by another type. A typical example of a dependent type is if we consider A to be the set of natural numbers and B to be a vector. The dependent type $B(x)$ then represents the family of types consisting of all vectors of length $x \in \mathcal{N}$.

As we saw that TTR records don't just have dependent types but dependent pairs, we introduce an additional piece of notation, namely the dependent pair type, also known as a dependent sum or just Σ -type.

Definition 3.8 (Dependent pair type). Given a piece of data $d : D$ and the dependent type C_d , we express the dependent pair type as follows.

$$\sum_{d:D} C(d) = (d : D) \times C_d \quad (9)$$

From the definition of the dependent pair, we can see that our TTR records are simply a chain of dependent pairs, also known as an iterated dependent pair, type telescope or just telescope [9], [89].

Definition 3.9 (Iterated dependent pair type). An iterated dependent pair type is a type that is defined by a sequence of dependent pairs. We can express this as follows.

$$(d_1 : D_1) \times (d_2 : D_2(d_1)) \times \dots \times (d_n : D_n(d_1, \dots, d_{n-1})) \quad (10)$$

Iterated or telescoping dependent pairs are generally used to model record types within the context of dependently typed proof assistants such as Coq. Naturally, due to their inherent dependent structure, they present less effective a candidate for representing the semantics of records in languages that aren't dependently typed.

Returning to the earlier work of Cardelli with the ζ -calculus, we saw fields being represented as functions. If we introduce this idea into DTT, we could view a record not as a set of pairs but as a function that we call with a label from a predefined set of

labels, which returns the value of a specific corresponding type. Thus, instead of using an iterated dependent pair, we can express this notion via a dependent product type. If we wish to express the collection of dependently typed functions that assign each $x : A$ to an element of $B(x)$, we can express a dependent product type as being formed out of a type A and a dependent type $x : A \vdash B(x)$.

Definition 3.10 (Dependent product type). A dependent product type denoted $\prod_{x:A} B(x)$ is a type that represents the collection of all functions that assign each $x : A$ to an element of $B(x)$. We can represent the formation of this type using the following rule.

$$\frac{\text{DEPENDENT-PRODUCT-FORMATION} \quad \Gamma, x : A \vdash B(x) : \text{type}}{\Gamma \vdash \prod_{x:A} B(x) : \text{type}}$$

For completeness, we can demonstrate what the application of a dependent product type looks like for a given input.

$$\frac{\text{DEPENDENT-PRODUCT-APPLICATION} \quad \Gamma \vdash f : \prod_{x:A} B(x) : \text{type} \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)}$$

This should hopefully make the idea clear that once we apply some input, we can see quite trivially the dependent nature that is being expressed here in the return type of the function f . From this, it might already be clear how we can apply this to formalize the idea of indexing into a record type. To get the last piece of the puzzle, we have to describe how indexing would work for an arbitrary input. That is to say, we want to derive a λ -abstraction that contains a term of type $B(x)$ and a free variable $x : A$. We can express this as follows.

$$\frac{\text{DEPENDENT-PRODUCT-INDEXING} \quad \Gamma \vdash x : A \vdash b : B(x)}{\Gamma \vdash \lambda x. b : \prod_{x:A} B(x)}$$

Finally, we can express the trivial β -reduction rule applied to the λ -abstraction.

$$(\lambda x. b) a \rightarrow_\beta b[x := a] \quad (11)$$

With this, we can now formally express the idea of a record type.

Definition 3.11 (Record Type). If we now assume that we have a set of labels defined by a type $L : \mathcal{U}$ and a family of types $T : L \rightarrow \mathcal{U}$. We can define a record type as a dependent product type indexed by the set of labels. By substituting the new notation for the created λ -abstraction, we can express the constructor for a record type as follows.

$$\Lambda L, T. \lambda L^L. \lambda \tau^{T_l} : \prod_{l:L} T_l \quad (12)$$

To illustrate how this lambda function arises, we can express the formation rule for dependent product types using more functional, yet equivalent, semantics.

$$\frac{\text{RECORD-INDEXING-FORMATION} \quad \Gamma \vdash L : \text{type} \quad \Gamma, l : L \vdash T_l : \text{type}}{\Gamma \vdash (l : L) \rightarrow T_l : \text{type}}$$

$$\frac{\text{RECORD-TERM-INTRODUCTION} \quad \Gamma, l : L \vdash \tau : T_l}{\Gamma \vdash (l \rightarrow \tau) : (l : L) \rightarrow T_l}$$

From this, it should be clear how the application then acts as just an instantiation of the record indexing process. Stated differently, we can now see how, at the type level, a record represents an abstract mapping between a label set and a type set, with instantiation then representing the actual, concrete mappings between labels of this set and values of the corresponding types.

To incorporate the idea of subtyping into our description, we can describe a high-level correspondence between the implementations of structural subtyping and nominal subtyping in the context of injectivity and surjectivity when viewed through a dependent type-theoretical lens. We will only superficially describe the idea of fiber as developing the full categorical semantics would be out of the scope of this work.

Definition 3.12 (Fiber of a Morphism). The fiber of a morphism or function $f : A \rightarrow B$ over a point B is the collection of elements of A that are mapped by f to this point. Informally, we can denote this as.

$$\text{fiber}(f, b) = \{a : A \mid f(a) = b\} \quad (13)$$

We can then proceed to define the concepts of injectivity and surjectivity in the context of dependent types. From this, we can form a connection between our formalism of record types and the idea of nominal and structural subtyping.

Definition 3.13 (Surjective Function). A function $f : A \rightarrow B$ is said to be surjective if, for all terms $b : B$, the fiber of f over b is inhabited. Here, we mean to express that for every element in the codomain, there exists an element in the domain that is mapped to it. On a high level, we can think of this as expressing the quantifier of existence or, in the set-theoretic sense, ‘at least one’.

$$\text{isSurjection}(f) := \prod_{b:B} \parallel \text{fiber}(f, b) \parallel \quad (14)$$

The notation $\parallel \text{fiber}(f, b) \parallel$ is the propositional truncation of the type expressing the fiber of f over b . For our purposes, it’s sufficient to understand this as expressing the predicate that the fiber of f over b is inhabited, which is to say that there exists some element or term in the domain that is mapped to b . In essence, it simply states the aforementioned quality of existence.

Surjection has a general correspondence to the more concrete idea of nominal inheritance-based subtyping. Specifically, we can express the idea that if there exists an abstract class A , then we will always have at least one concrete class B that is a subtype of A . A key observation to note is that surjection enables us to map multiple elements, in this case, subclasses, to a single element, the superclass. A byproduct of this quality is that we lack the property of uniqueness. We can view this as analogous to the fact that in nominally typed languages, unless we perform a, usually runtime,

downcast, an object only has those properties that are defined in its superclass.

Definition 3.14 (Injective Function). A function $f : A \rightarrow B$ is said to be injective if, for all terms $b : B$, the fiber of f over b is a sub singleton. By which we mean to express that the collection of elements A which are mapped to b consists of either a single element or no elements at all. Here again, on a high level, we can think of this as expressing the quantifier of uniqueness or, in the set-theoretic sense, ‘at most one’.

$$\text{isInjection}(f) := \prod_{b:B} \prod_{a:A} f(a) = b \quad (15)$$

Injection then has a correspondence to the idea of structural subtyping. Structural subtyping, often seen more in dynamically typed languages, relies on implicit structural comparisons to express the idea of substitutability. The key distinction of nominal subtyping is that we preserve the unique quality of objects; in practice, this idea is often leveraged to allow for more flexible uses of existing implementations. The uniqueness of object mappings means that unlike in the case of nominal subtyping, where we typically have to downcast to access the complete set of properties of an object, in the case of structural subtyping, excess properties are preserved and at least in theory, can be accessed without any downcasting as there is a unique mapping back to the original object which preserves the structure.

Of course, this is an oversimplification, as type systems don’t necessarily implement these ideas in mutually exclusive ways or even in the most direct way. Nonetheless, for the applications of our extension, we can utilize these ideas to develop a more precise language for discussing the steps taken to extend the Jlua library. To then summarize and elaborate on how we can proceed, we have now developed a basic way we can more precisely use to speak about record types, the primary focus of our extension; furthermore, we have a way of understanding how we can express subtyping in terms of the same theory we used to talk about record types, namely dependent type theory. We have also established, more broadly, that metaprogramming and binding generators, for that matter, can be understood as partial evaluators on some static specification. Using this groundwork, we can build some key ideas that we can directly apply in the construction of our extension. These ideas, or takeaways, will be developed in the next section.

We can first review the record type again; specifically, let’s break down what we consider the field type.

Definition 3.15 (Field Type). A field type is the dependent type, which is indexed by the set of labels. We can interpret this as the function we formed in the record indexing formation.

$$(l : L) \rightarrow T_l \quad (16)$$

This field type provides an abstract view of the more concrete mapping we think of as labels for values of a particular type. When we instantiate this type, we specialize it to a specific, concrete mapping of labels to values. Alternatively, more accurately, a compiler or interpreter would specialize this type to a concrete mapping of labels to values. If we revisit the analogy between a binding generator as an interpreter and put ourselves in the shoes of this binding generator, then a field type, as shown above, would be considered

already specialized for some set of labels and types. A question that we must ask ourselves, then, is what type of field this is.

Let's first consider again what a type is. From a high-level set-theoretical sense, we can think of a type as characterizing a set of elements belonging to this type. Alternatively, as demonstrated by the Curry-Howard Correspondence, we can understand types as being constructors that encode some set of propositions and terms of this type as being proofs of these propositions. A common informally shared aspect, regardless of interpretation, is that types, in a sense, are just abstract descriptions. To then get back to the question at hand, the type of the field type is, abstractly speaking, simply a description of the field type itself. By observation alone, we can create an informal description of the properties of this type.

- (1) *Label*: We can confidently say that all fields clearly must have a label $l \in L$.
- (2) *Output*: We can also see that field types have some dependent output type $T(l)$.
- (3) *Record type*: Each field is also associated with a record T .

Informally, a natural way we might then represent a constructor for this type could be as follows.

paper/code/constructorex.py

```
1 class FieldType[Label_t: str, Out_t, Rec_t]:
2     label: type[Literal[Label_t]]
3     out_t: type[Out_t]
4     rec_t: type[Rec_t]
```

Listing 6: Field type constructor pseudocode

A few notes we can make here is that firstly, as labels are generally strings, we can express them as being concrete value types bounded by strings. Concrete value types bounded by a more abstract type, such as strings or integers, are commonly also called literals and can be understood as a form of static type narrowing. Secondly, as we are working at a higher level of abstraction, the types of the attributes of field types are descriptions of the underlying types, which are expressed via the 'type[T]' notation, representing the type of the type.

A key issue we find with this is that while we have described a kind of recipe for how we might represent the type of a field, currently, the description in 6 does not tell us anything about how we are supposed to *use* a field. So, we need to expand this description; the somewhat trivial question that then appears is, what does it mean to use a field type? We can quite easily describe this as an operation to retrieve a value from the field and a way to set a value in the field. Therefore, it would be reasonable to think that we want to expand our description to include two functions: a getter and a setter. To be more rigorous, we can express these functions as follows.

Definition 3.16 (Field Accessors AKA Lenses). We can describe the getter of a field as a function that takes a record of type T and returns the value of the field in the record defined by T_l . A setter, conversely, is a function that takes as input a record and a value of type T_l and returns a new record with the value of the field set to this value.

$$\text{get} : T \rightarrow T_l \quad (17)$$

$$\text{set} : T \times T_l \rightarrow T \quad (18)$$

Not surprisingly, the notion of lenses has already been extensively studied in various contexts. With their origins being traced back to a solution of the view-updated problem presented by Foster et. al. [30]. Commonly, this is then placed in a more general context within functional programming to describe the aforementioned idea of just getting and setting values. The abstract idea of different forms of data accessors is generalized to the concept of *families of optics* of which lenses are a well-known part. [18]

In our case, for the case of field getters and setters, it's sufficient to constrain ourselves to the notion of lawful lenses.

Definition 3.17 (Lawful Lenses). Since we don't want our getters and setters to behave arbitrarily, as that would be absurd, we can define the set of laws that we expect our lenses to follow. Specifically, we can enumerate what is defined as a lawful lens. Three main rules characterize a lawful lens [30]. If we consider a source S (analogous to our record type) and a view V (analogous to our field type), we can define the following laws. One note on notation is that different pieces of literature use different notations for getting and setting. In this case, we will use the notations get and put.

$$s : S \vdash \text{put}(\text{get}(s), s) = s \quad (\text{Get-Put})$$

$$v : V, s : S \vdash \text{get}(\text{put}(v, s)) = v \quad (\text{Put-Get})$$

$$v_1, v_2 : V, s : S \vdash \text{put}(v_2, \text{put}(v_1, s)) = \text{put}(v_2, s) \quad (\text{Put-Put})$$

In plain English, the rules translate to the following.

- (1) *Get-Put*: If we read a value and then write this value, there should be no change.
- (2) *Put-Get*: If we write a value and then read a value, we should get the value we wrote.
- (3) *Put-Put*: If we write a value and then write another value, we should only get the last value we wrote.

The first two qualities describe lenses which are *well-behaved* with the inclusion of the third rule giving us a *lawful* or *very well behaved*⁵ lens.

With this, we can now expand our description of the field type to include its applications. Building on our previous example, we arrive at the following constructor. For clarification 'Callable[[A], B]' is equivalent to $f : A \rightarrow B$.

As this pattern is based on well-studied and somewhat intuitive principles, it forms the foundation for various types of wrapper types or type specifications, which aim to find an abstract way to characterize and constrain the properties and behaviors of types. We can see this in the original implementation of JIuna in 8.

In the case of JIuna, the upper template parameter T is that of the abstract record type; in JIuna, these are called `User type<T>`. As the name implies, `Field_t` is the type of the field; in our case, this corresponds to T_l . The function then takes three arguments: a name of the property, i.e., the label, the getter `box_get`, and the setter

⁵Work discussing lenses with the generalized context of optics tends to consider exclusively lawful lenses. [18], [62]

```
paper/code/constructorex2.py
```

```
1 class FieldType[Label_t: str, Out_t, Rec_t]:
2     label: type[Literal[Label_t]]
3     out_t: type[Out_t]
4     rec_t: type[Rec_t]
5     get_t: type[Callable[[Rec_t], Out_t]]
6     set_t: type[Callable[[Rec_t, Out_t], Rec_t]]
```

Listing 7: Expanded field type constructor pseudocode

```
jluna/.src/usertype.inl
```

```
1 template<typename T>
2 template<typename Field_t>
3 void Usertype<T>::add_property(
4     const std::string& name,
5     std::function<Field_t(T&)> box_get,
6     std::function<void(T&, Field_t)> unbox_set
7 );
```

Listing 8: Jluna add property function

unbox_set. An important takeaway from this example is that since all information is statically known, the compiler specializes in all property specifications of T at compile time.

Given that we now have a way of expressing record types and an informal way of describing their fields, we can use the idea of lenses introduced to create a more precise descriptor of how we would express the type of a record type. The first concept we introduced when discussing formal semantics was the dependent product type. Just as we applied this as a means of defining a record structure, so too can we apply it to define the structure of a lens. We can delineate the structure of our lens into three parts.

- (1) *Type*: The type on which the lens operates.
- (2) *Semantics*: The semantics of the lens, *i.e.*, the getter and setter.
- (3) *Laws*: The laws which the lens must follow, *i.e.*, the lens must be lawful.

To express this formally, we can define the lens type as follows.

Definition 3.18 (Lens Structure). Given a dependent T_l indexed over a set of labels L , we can define the structure of a lawful record lens for this type as follows.

$$\text{Lens}_L = \begin{cases} T : (l : L \vdash T_l) : \text{type} \\ \times (\text{get}_{T_l} : T \rightarrow T_l) \\ \times (\text{set}_{T_l} : T \times T_l \rightarrow T) \\ \times (\text{rw} : (x : T_l, \tau : T) \rightarrow \text{Id}_{T_l}(\text{get}_{T_l}(\text{set}_{T_l}(x, \tau)), x)) \\ \times (\text{wr} : (\tau : T) \rightarrow \text{Id}_T(\text{set}_{T_l}(\text{get}_{T_l}(\tau), \tau), \tau)) \\ \times (\text{ww} : (x_1 : T_l, x_2 : T_l, \tau : T) \rightarrow \text{Id}_T(\dots))^6 \end{cases} \quad (19)$$

A new construct introduced here is the concept of the identity type ‘ Id_T ’; this represents the notion of equality between two terms of type T . Hence the inclusion of the functions ‘wr’, ‘rw’ and ‘ww’

act as laws over the type T . In other words, we are encoding the laws that we want our lens to follow into the structure of the type over the records. [56]

With this, we have a baseline for discussing the structure and dynamics of our record type from the more abstract perspective that we work with in the case of binding generators. Particular relevance in our definition of the Lens is the base type, *i.e.*, the abstract record type. The abstract record type represents the basic language-agnostic model of objects, with both languages needing to conform in order to be bridged. There are various approaches to ensuring conformance to an abstract model, which typically depends on the languages being bridged. A fundamental approach if we have compatible languages is to generate bindings in a conformant manner as a preprocessing step via header parsing. In this instance, a binding generator would typically either ensure that the generation process is implicitly conformant and expose idiomatic types, or it would generate wrappers that offer the aforementioned added flexibility at the cost of idiomatic usage, depending on the implementation’s performance. The approach taken by Jluna is to generate idiomatic bindings where the conformance rules for the type structure are defined implicitly through the application of the type specification in the runtime generation process. Julia enables this through its implementation of metaprogramming, allowing the definition or specialization of a set of types at runtime based on an abstract description of their different properties.

3.2.2 Abstract Types & Substitutability. In our case, there are two key interrelated things we wish to extend the library with. Firstly, support for abstract types, and from this, support for concrete types that implement this abstract representation. Abstract Data Type’s (ADTs) in the theoretical sense means to express a type which encodes a contract or model for a set of behaviors and properties which a concrete variant of this type must adhere to. A basic example could be the abstract definition of a list, which is commonly parameterized with a type argument representing various concrete variants, such as a list of ints or a list of strings. The common semantic properties then include the ability to add, remove, or access elements. A somewhat trivial benefit of this is that unless we must know the concrete type of a list, we can work with this abstract definition and, hence, write more generic code that can work for a variety of concrete variants. Polymorphism is a natural byproduct that comes from the idea of substitutability of ADTs. If an ADT represents an abstract contract with a set of concrete types model, then we can say that any type following this contract could reasonably be substituted in place of this ADT without any behavior change; this is the general idea of polymorphism or, more specifically, behavioral subtyping. Liskov formalized a strict condition of this idea in her Substitution Principle.

Definition 3.19 (Liskov Substitution Principle). We denote $\phi(\tau_T)$ as a property provable about objects τ_T of type T . Then $\phi(\tau_S)$ should be true for objects τ_S of type S where S is a subtype of T denoted $S <: T$. [50]

$$S <: T \rightarrow (\forall \tau_T : T) \vdash \phi(\tau_T) \rightarrow \phi(\tau_S) \quad (20)$$

Due to its strictness, the condition is often relaxed in subtyping implementations. Nonetheless, it serves as a good basic formalism for considering substitutability.

One common practical instance in which we see behavioral subtyping occur is in the case of interfaces. Interfaces within the context of programming languages refer to implementations of the aforementioned idea of an abstract type. Behavioral subtyping is typically observed in the dynamics of interface implementation. In the case of many nominally typed object-oriented programming languages, this takes the form of an abstract class representing the interface and an inherited concrete class that forms the implementation. It should be noted that while inheritance can allow for subtyping, it does not necessarily imply it. Likewise, the presence of subtyping does not imply inheritance. Another paradigm that has emerged as a solution to the deficits associated with inheritance-based subtyping is the idea of composition-based subtyping, with the familiar phrase being ‘favor composition over inheritance’. Composition-based subtyping is similarly based on the idea of an abstract interface, which dictates a set of behaviors and properties that a concrete implementation must follow. The key distinction is that a concrete implementation does not inherit from the abstract interface but rather is composed of it. In other words, for composition-based subtyping, the concrete implementation exists as an instantiated version of the abstract interface directly rather than an inheriting subclass. Implementations of composition-based behavioral subtyping are often expressed in the form of protocols, traits, or interfaces, which serve as a contract that a concrete implementation must adhere to. To clarify again, the difference here is that with composition-based subtyping, there is no hierarchical relationship between the abstract interface and the concrete implementation; the concrete implementation is simply a realization of the abstract interface. Composition-based subtyping can be understood more broadly as an implementation of parametric structural subtyping or, more simply, generics, in which substitutability is determined by the structure of the object, which exists as a specialization of the general interface.

A crucial point of consideration is that since subtyping is a function of a language type system, how do its enforcement properties influence the dynamics of subtyping? We can categorize enforcement into two main aspects: when enforcement occurs and how it occurs. Since we are explicitly concerned with substitutability, the question becomes: when do languages enforce this, and how do they do so? When it comes to the ‘when,’ the trivial approach would be to delineate compile-time and runtime, though as JIT compilers, in a sense, combine the two, it makes more sense to state it in terms of information availability. We can define a hypothetical object o of type S . We have a method taking an argument of type T where $S <: T$, the most trivial variant of substitutability would occur in the case where $S = T$, and this is precisely what we tend to see with parametric structural subtyping, here methods are specialized such that for all concrete variants of a type T we have a corresponding concrete method accepting this variant. For statically typed languages, we see this as generic methods being instantiated through means such as the aforementioned monomorphism, although it can also occur using runtime-based approaches. In the case of dynamically typed languages, there is a little more nuance depending on implementation; the untyped case, which is purely structural subtyping, is defined by the semantic properties of how the argument is applied. The typed case refers to dynamic languages that have a gradual type system. Here, there is considerable variation in not

only the execution method (interpreted, JIT compiled) but also in the extent of enforcement and application. At one end of gradual typing systems, we have TypeScript, which exists in large part separate from the underlying execution of JavaScript and simply acts as a layer on top of JS.

On the other hand, we have languages like Julia, which, despite having a form of gradual typing, make considerable use of type annotations as a means of more effectively specializing code to minimize runtime compilations. Nominal subtyping, on the other hand, tends to be a predominantly runtime process, regardless of the language, the main reason being that with nominal subtyping, we express substitutability as a function of an explicit hierarchy, as opposed to direct static equality. The hierarchy is then something that must persist and be used for us to determine a valid substitution as opposed to it being trivially and apparent. A notable distinction is that statically typed languages enforce, at compile-time, that abstract methods be implemented in an inheriting class, such that LSP does not end up being violated. In contrast, dynamically typed languages might have their type systems enforce a nominal hierarchy via annotations, but this does not necessarily imply that LSP is met for all properties. The basic takeaway from this is that we can interpret type systems as existing on two spectrums: the extent to which they are integrated and applied within the language runtime and the strictness of the invariants they enforce.

3.2.3 Substitutability in Julia and C++. What we see in Julia is that it enforces nominal subtyping of abstract types in only the most direct sense, which is to say, for a type S and an abstract type T , as long as there exists a declared subtyping relationship $S <: T$ then we can use S in place of T . The declaration has no enforcement on the actual underlying structure of the object; here, Julia falls back to simple error checks if property access does not resolve. Within Julia, subtyping enforcement is entirely a function of the type declaration, as opposed to any structural qualities of the object. We can see this in more rigorous terms through the work of Zappa et al. [87], which aimed to formalize Julia’s implementation of subtyping.

Definition 3.20 (Julia Subtyping). As defined by Zappa et al., the type-level representation of a user-defined type can be denoted as follows.

$$\text{attr name}\{t_1 <: T <: t'_1, \dots, t_m <: T <: t'_m\} <: t'' \in \text{tds} \quad (21)$$

Here, *attr* expresses if a type is concrete or abstract, *name* is the name of the type. Following this, we have the type parameters T each, which have their defined lower bound t_i and upper bound t'_i , in addition to any type bounds applied to the type itself t'' . The type definition is then a part of global type declarations *tds*. The inference rule for application of subtyping is then defined as follows for a given typing context Γ

$$\begin{array}{c} \text{APP-SUPER} \\ \frac{\text{name}\{T_1, \dots, T_m\} <: t'' \in \text{tds} \quad \Gamma \vdash t''[a_1/T_1 \dots a_m/T_m] <: t' \vdash \Gamma'}{\Gamma \vdash \text{name}\{a_1, \dots, a_m\} <: t' \vdash \Gamma'} \\ \\ \text{APP-INV} \\ \frac{n \leq m, \Gamma_0 = \text{add}(\text{Barrier}, \Gamma), \forall i \in [0, n] \quad \Gamma_{i-1} \vdash a_i <: a'_i \vdash \Gamma_i \wedge \Gamma_i \vdash a'_i <: a_i \vdash \Gamma_i}{\Gamma \vdash \text{name}\{a_1, \dots, a_n\} <: \text{name}\{a'_1, \dots, a'_n\} \vdash \text{del}(\text{Barrier}, \Gamma_n)} \end{array}$$

Here, the rule APP-SUPER expresses the application of a subtype to a supertype, and the rule APP-INV expresses the application of a subtype to an inverse subtype. APP-INV introduces some new notation, namely *add* and *del* of a ‘Barrier’; a ‘Barrier’ represents a tag used to delineate or distinguish between type variables introduced before and after the constructor. The check itself then ensures the constructor application is invariant.

The case of C++ is more complex, as it’s a statically typed language. Aside from enforcing a nominal hierarchy like Julia, it also enforces a structural hierarchy to avoid violating LSP ahead of time. Informally, we can say that given a type *S* and an abstract class of type *T*, for *S* to inherit from type *T*, it must implement all the abstract methods of *T*, in a sense enforcing that it’s a valid substitution. A key difference between this and structural subtyping, or some variants of parameterized structural subtyping, is that in the abstract context, we lose any excess properties of the object. Stated differently, inheritance-based subtyping overlaps with parameterized structural subtyping if it enforces a concrete implementation of abstract methods, with the difference being that we lose trivial access to excess properties (*i.e.*, we need downcasting).

4 CASE STUDY: POLYMORPHISM IN A MULTI-LANGUAGE RAY-TRACER

In the previous sections, we discussed some of the theoretical underpinnings of language interoperability and their relationship to the practical aspects of implementing cross-language tooling. A key emphasis of our exploration, especially in the latter half, was comparing and contrasting the attributes of interoperability tooling between similar and dissimilar languages and their respective runtime models. The idea of reconciling the semantics of different languages and leveraging their respective strengths is, in large part, what motivated the exploration of this topic. Julia, especially, is an interesting candidate for the case study, as its performance characteristics are considerably more dependent on an idiomatic programming style than those of most other languages. Idiomatically written code, regardless of the language, tends to be more performant, as the set of paradigms it applies tends to enable a more effective use of the underlying execution model. Phrased differently, we can understand idiomatic code to be defined by a set of soft or non-strictly enforced requirements as defined by the language execution model. Julia’s execution model, the JIT compiler, imposes a more nuanced and impactful set of requirements on idiomatic code, which in turn can lead to a higher learning curve for new users.

One of Julia’s main selling points is that *when written well*, it has performance characteristics close to that of C while retaining the high-level ease of use of Python. The somewhat unique nature of Julia, or more specifically, its JIT compiler, is what served as the primary motivation to use a ray-tracer implementation written in Julia as the main testbench for this exploration. The ray-tracer itself is a relatively simple implementation based in large parts on Peter Shirley’s guide [59], [60] and his C++ implementation. The initial Julia implementation exhibited performance characteristics that were less than ideal when compared to Peter Shirley’s equivalent C++ implementation. To elaborate on why the non-idiomatic nature of the implementation led to such stark performance differences,

we will briefly explain how Julia’s JIT compiler works. A quote from the Julia forums summarizes this nicely.

... in short, it’s a reasonably good static compiler but a terrible JIT

The first response some people might have to this is confusion. Julia is a JIT-compiled language, so how is its JIT compiler terrible. Well, this comes down in large part to how its JIT compilation works. The core of this is how it performs online partial evaluation (OPE) concerning its type system. OPE w.r.t, the type system in this instance, refers to the process of, at runtime (online), Julia specializes (partially evaluates) methods for which it can infer the types of every local variable from the types of the arguments or any other constants; in this instance, a method is said to be ‘type stable’. Specializing type stable methods means it compiles methods for the set of concrete types it can infer. Practically speaking, this mechanism can also be viewed as a form of monomorphizing a set of functions for which the JIT compiler can determine the concrete types, thereby caching their compiled instances for fast execution. The reason, then, why Julia’s JIT compiler could be called ‘terrible’ would be that it, currently at least, does not implement profile-guided optimization (PGO). The lack of PGO and its reliance on type-stable code for speed means that there is a strong correlation between idiomatically written Julia code and its performance.

Naturally, there are various approaches one can take to deal with a situation like this. First and foremost, this would generally involve writing the code in a type-stable manner, which in turn would optimize performance by reducing the number of cache misses on precompiled methods. However, a case could also be made for simply using existing high-performance code, which avoids rewriting altogether. In practice, using existing high-performance libraries in dynamic languages, especially in the context of scientific computing, usually means using libraries that wrap, often via FFI, fast, low-level frameworks like those that implement the Basic Linear Algebra Subprograms (BLAS) specification for everyday linear algebra operations. An idea that arose from this was to replace the most performance-intensive component of the ray tracer entirely with implementations in C++ and Python, rather than simply using existing libraries to replace slower operations. The idea here is twofold: firstly, we can assess the viability and performance characteristics of interoperability when applied to a more complex system. Secondly, we can also compare and contrast the earlier discussed relations between language similarity and overhead.

4.1 Project Context & Toolchain

As one of our objectives is not only to research the theoretical underpinnings of interoperability but also to assess their viability in practice, we now describe how many of the previously discussed topics are relevant in real-world scenarios. In the preamble, we decided to use a ray-tracer implementation written in Julia as the main subject for this exploration. Before delving into the implementation of cross-language aspects, it’s worthwhile to provide some motivation for why a ray tracer serves as a suitable subject. Although the decision was initially ad hoc, it presents several potentially insightful attributes. Ray-tracers, especially more developed legacy implementations, are well-known for their complexity—not only due to their foundation in mathematical modeling but also

because of the interconnectedness of various subsystems required for image generation. Even minimal ray-tracer implementations present a situation analogous to more complex systems, offering broader insights into the effects of interoperability compared to domain-specific minimal examples.

Performance is another key aspect of ray tracers. While their origins lie in drawing simple images, modern ray tracers—or, more broadly, physically based rendering (PBR) frameworks—are used in real-time in video games, creating a significant demand for high-performance PBR systems. Although the ray-tracer used in this paper implements only a minimal subset, it directly relates to a domain where performance is critical, serving as an indicator of the viability of the applied interoperability techniques in performance-demanding scenarios. Finally, despite the complexity of ray tracers, they tend to be modular systems in certain respects. This modularity allows us to more easily separate logically distinct components, making the rewriting process more manageable. A high-level overview of the ray tracer can be seen in Figure 2.

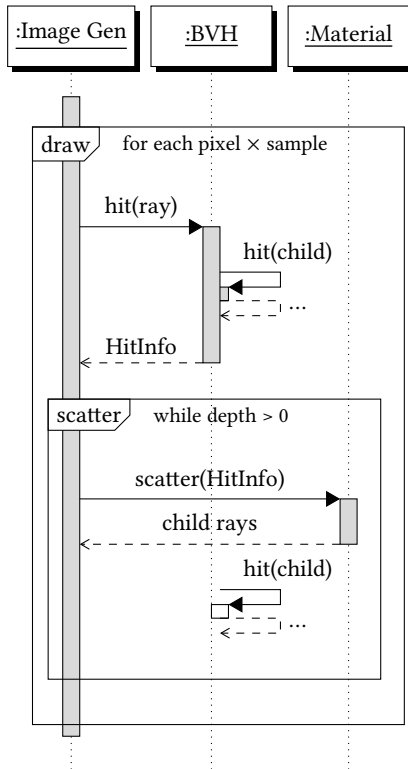


Figure 2: Ray-Tracer high-level control flow

4.1.1 Implementation approach. Now that we have established our motivation for testing and have an implementation to work with, the next step is figuring out how to test it in practice. To isolate the most performance-impacting component, we profile the application, analyzing both runtime and memory usage before making informed judgments based on this data. Regarding language choices for rewriting, our constraints are those with embedding APIs for Julia, but we still have several options. It makes sense to choose two languages with fundamentally different properties—Python and C++—as they provide a broad spectrum of insights applicable

to similar domains. Both languages have well-established interface libraries, making them practical choices for evaluation. To test the effectiveness of the component, we use benchmarking tools to analyze its performance in isolation, along with Julia-sided tools to assess the combined performance of the component, interface code, and any overhead introduced.

4.1.2 Choice of Binding Generators. Writing the components alone is not enough if we can’t integrate them into the larger system, so the next step was selecting an embedding API to attach the hit functions to the ray tracer implementation. For Python, the choice was straightforward—Python/JuliaCall [65] was used, which builds on an earlier implementation and supports a broader range of conversions between the two languages. C++, however, presented a more interesting challenge. Unlike Python, which has trivial reflection mechanisms making object conversion straightforward, C++ requires more deliberate choices. The two main options were CxxWrap [44], which has a strong feature set but lacks user-friendliness, and Jluna [22], a newer and more user-friendly alternative that is missing some mapping mechanisms. We chose Jluna because the focus was on minimizing additional code and effort while creating usable components in another language. Jluna, leveraging modern and extensible C++ techniques, proved to be more future-proof and developer-friendly. We can tie the future outlook to a broader consideration: when rewriting a component in another language, if the results are positive, ideally, we want minimal refactoring of our existing codebase to integrate it into the larger system. This means choosing an API that (1) requires the least additional code and (2) minimizes the code needed to attach the component. One key takeaway from this process, and something which might already be quite evident given the discussion in 2, is that working between Python and Julia already poses significantly fewer challenges when searching for a binding generator due to the added feature set provided by both languages being designed with dynamism—and by extension runtime reflection—in mind.

4.1.3 Implementation setup. The code for setting up the scene, which includes parsing the ‘.obj’ files that describe the vertices that bound all faces in the scene, loading the faces into a bounding volume hierarchy (BVH), and managing the actual image generation by drawing the pixels, is all uniformly done in Julia. The isolated components, namely the tree search and hit computation for a given ray cast, are rewritten in C++ and Python and then replaced in the Julia implementation, depending on which language is used to initiate the ray tracer. In both instances, we redefine the hit functions such that the Julia code is redirected to the language-specific variants, which act as proxies to the actual host language code.

4.2 Design Challenges

4.2.1 Problem Statement. Now, while the implementation here was straightforward in theory, a significant issue encountered was related to the constraints imposed by the binding generator between Julia and C++. To explain the cause of this issue, it’s helpful first to outline what the implementation needs to do, or more specifically, what we are moving across the language boundary. As the BVH traversal served as the most logical component to isolate due to

its aforementioned separation from other aspects, we chose it to serve as the bridge. The three main things needed in our implementation when trying to do hit detection are the BVH, which spatially structures the objects in the scene; the actual ray for which we are computing the hit; and finally, a hit record, which stores the information from the hit (e.g., material, normal vector to the surface, etc.). Assessing the complexity of these types, the simplest one is the ray, as it just stores an origin and direction. Next is the hit record, which has polymorphic attributes, namely the material of the object that the ray hit. Finally, there is the BVH tree, which, as implied in the name, is a tree structure that adds another layer of complexity, as there are now recursive polymorphic attributes with the left and right child being BVH nodes, which are subtypes that define the underlying type of the shape represented (e.g., Triangle, Sphere, etc.).

In both instances, the type of polymorphism being discussed is subtyping polymorphism. While both C++ and Julia, abstractly speaking, implement similar conventions for enabling subtyping polymorphism—namely, some abstract type and a means of specifying a nominal type hierarchy—there are considerable differences when it comes to the practical case of interoperability. A key distinction between the high-level notions of interoperability and the low-level practicalities is how types are represented. Since we are fundamentally working through the C-ABI, or more accurately, through an abstraction (Jluna) over the Julia C-API, before we do anything, we need to parse the underlying representation of Julia types. Fortunately, this process is elegantly implemented within Jluna for all types, including structs, by the original author. Jluna achieves this mainly by using generics to deconstruct the low-level representation of a Julia type recursively. To ensure constraints on the implementation—by which we mean rejecting types that cannot be parsed for some C++ analog—the implementation utilizes constraints (concepts, in this case) on its generic deconstruction methods, which ensure certain invariants hold that constrain the set of bridgeable types to only those specified by the programmer. One of these constraints was a static assertion that prevented bridging abstract types—in other words, types that exhibited subtyping polymorphism—which brings us back to the issue we encountered. The question then becomes: How do we solve this problem? Trivially, a straightforward solution to the C++ and Julia interoperability would have been to rewrite the ray tracer such that the BVH and hit record represent everything in a non-polymorphic format. However, there are a few reasons we were motivated to bridge these types.

- (1) *Challenging Common Notions*: it’s often assumed in the context of interoperability for larger organizations that the set of bridgeable behaviors should be restricted to minimal constructs. We sought to challenge this assumption; as type systems in mainstream languages become increasingly expressive, a highly restrictive approach to cross-language systems may eventually limit the applicability of libraries that leverage more advanced type system features.
- (2) *Flexibility Discrepancy*: Additionally, we aimed to clarify whether the observed discrepancy in flexibility between dynamic and statically compiled languages in cross-language

systems is inherent to the nature of type systems and execution models or if it’s primarily a consequence of specific implementation philosophies or limited use of language features.

- (3) *Viability of Loosening Constraints*: We were interested in evaluating the viability of relaxing constraints, both in terms of the resulting implementation complexity and the performance implications associated with bridging such types.
- (4) *Feasibility Exploration*: Finally, we aimed to explore the overall feasibility of complex type bridging, given its relative rarity in cross-language systems involving statically typed languages—particularly when the statically typed language is the primary focus rather than the dynamic language. We anticipated that this exploration could yield valuable insights.

To contrast this with the case of Python and Julia, the story was interesting in its own right. JuliaCall, the library we chose for bridging between these two languages, wraps Julia types with opaque `juliacall` class objects that idiomatically mirror their Julia counterparts in terms of field access. The use of opaque wrappers, in turn, means that unless we implement custom conversion rules to produce the actual class objects corresponding to those we defined within the Python implementation, we don’t have any subtyping relationship. We can still emulate nominal subtyping by overloading python’s implementation of the `singledispatch` decorator using unique properties the opaque wrapper provides, though this is a rather cumbersome solution.

With the broad idea in mind of what we want to achieve now and some motivation as to why, it makes sense to establish a more concrete approach. Before we assess the details for how to extend the Jluna library, we should first assess in detail how the library works currently. As discussed throughout 2, if we think of C++ as our host language and Julia as the guest, then boxing would be the operation of representing a C++ type in Julia, and unboxing would be the reverse operation. In Jluna, boxing and unboxing have multiple steps; we can give a high-level overview of it as follows, with a general breakdown of what each step entails.

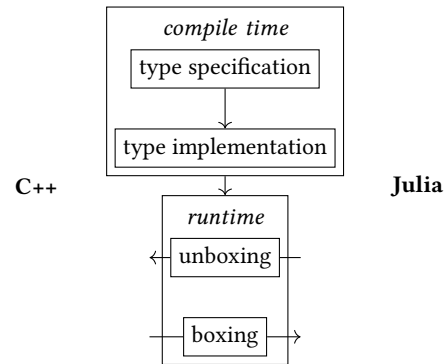


Figure 3: High-level overview of Jluna application

- (1) *Type specification*: For the type specification of object types (i.e., classes and structs), the programmer defines a set of getter and setter functions in addition to some

other type attributes, which are then encoded in a meta-type akin to meta-types generated via dynamic reflection mechanisms in dynamic languages.

- (2) *Type implementation*: Once this type is defined, the programmer calls an implementation method that utilizes the meta-type to generate an idiomatically mirrored Julia type using its metaprogramming features, which creates the corresponding type within the Julia runtime.
- (3) *Unboxing*: Unboxing is then applied to all arguments of methods that are exposed as cross-language via an implicit unboxing proxy, which is generated through the specification of the function type signature in the Jluna framework.
- (4) *Boxing*: Conversely, Boxing is applied to all return values of the function, which are then passed back to the host language, again as an implicit wrapper around the bridged function.

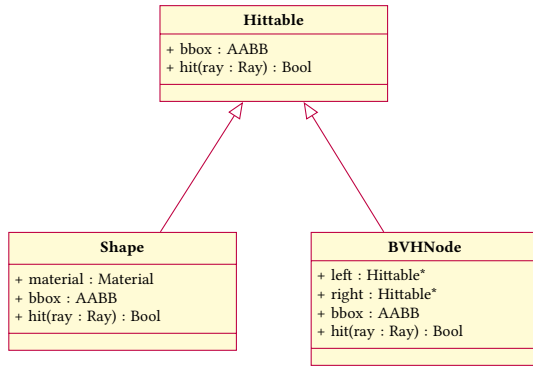


Figure 4: Simplified class diagram of the BVH tree structure and its polymorphic components.

With this in mind, we can already make a few key observations. As stated before, the library is designed to disallow abstract types entirely; hence, this would likely form the first step to our extension. Furthermore, as this library was not designed with abstract types in mind, it naturally did not implement a way to accommodate such types or their attributes in the type implementation (Julia instantiation), boxing, or unboxing. From this, a few interesting questions arise as to how we proceed.

- (1) *How do we encode the abstract base class?* Given that the library was designed to disallow abstract types fully, the type specification needs to implement a mechanism to express subtyping relationships.
- (2) *What are the effects of this constraint on the implementation of the library as a whole?* More broadly, this question of extensibility in library design often appears. One interesting aspect of Jluna, specifically, is its heavy usage of compile-time metaprogramming and highly generic type encodings, as opposed to, for example, concrete wrappers or more traditional parsing approaches, as a means of generating bindings. A natural question that then arises is how the extensive application of these paradigms affects the ease of extensibility, given the constraints placed on the initial implementation.
- (3) *Unboxing - How do we parse the abstract Julia type in C++?* Naturally, as the nature of our problem is bidirectional, we

need to reconcile how the low-level BVH tree representation in Julia is parsed in C++.

- (4) *Boxing - How do we represent parse the abstract C++ type to create the corresponding Julia type?* Similarly, we need to ensure that the abstract C++ type is not only implemented in Julia but that we also have a means of moving the C++ type to this corresponding implementation.
- (5) *Constraints - How do we loosen constraints while retaining safety?* We need to ensure that we can represent arbitrary BVH trees and establish a method to relax the constraints on type specifications without compromising the system's safety. In other words, we should define a new set of guidelines for what qualifies as a valid polymorphic type specification.

4.2.2 Metaprogramming in Jluna. Before elaborating on the actual code that implements this extension, there are two points we find important to establish in more detail before moving on. We have mentioned several times the use of compile-time metaprogramming in the Jluna library. While we have provided a high-level overview of what this entails, we want to dedicate some time to explaining certain aspects of this design paradigm in the original implementation. Additionally, we will examine how these elements were utilized to address some of the questions that emerged during the development of the extension. We begin by providing a brief overview of templates in C++ and how they are used in the Jluna library.

libs/jluna/.src/usertype.inl

```

1  template<typename T>
2  bool Usertype<T>::is_implemented() {
3      return _implemented;
4  }

```

Listing 9: Jluna implemented query function

In 9, we see the most basic application of a template, one that should be familiar to most programmers who have used generics before. In this instance, for the set of specified types $T \in \mathcal{T}$, we instantiate the function for each type in the set. A few observations to make here is that the type T in this isolated instance is not bounded, which is to say that, ignoring `Usertype`, this function could be instantiated or specialized with any arbitrary single type. To allow for some level of restriction in the types that are permitted to specialize a template, C++ implemented its version of constrained generics called concepts. Concepts define compile-time predicates which express a requirement on a template argument. We can see a basic application of this here.

libs/jluna/include/usertype.hpp

```

1  template<is_double> T>
2  unsafe::Value* box(T value) {
3      return jl_box_float64((double) value);
4  }

```

Listing 10: Jluna concept function

In this case, we use a concept to call the most appropriate variant of the box function for the double type. Concepts enable static method dispatch; in other words, C++ uses constrained parametric polymorphism to emulate the behavior of dynamic dispatch, as the constraints define a subtyping relationship. Naturally, the language extends this idea to multiple types of arguments, each with its own set of constraints. A question that might arise from seeing this language definition is: What if we want to define a template with a set of types of arbitrary length? Generics that accept an arbitrary set of types are known as variadic generics, also referred to as variadic templates or template parameter packs in the context of C++. Furthermore, the concepts can also be applied to these variadic templates. Once again, we can see an example of this being applied in the Jluna library.

```
libs/jluna/include/proxy.hpp
```

```
1 template<is_boxable... Args_t>
2 Proxy safe_call(Args_t&&...);
```

Listing 11: Jluna variadic concept function

On a high level, all that’s happening in 11 is that we are checking the predicate that for each member of `Args_t`, which represents an arbitrary length set of function arguments, the type of this argument is boxable and hence transferrable across the language boundary. In this snippet, we see two new pieces of notation introduced, namely ‘...’, which is used to denote a variadic generic, and ‘&&’, which in this represents an *rvalue reference* which is a reference that binds to an rvalue (temporary value) which is about to be destroyed enabling move semantics for all arguments passed to the function.

A final relevant construct worth mentioning is the template metafunction, a basic example of which can be seen in Listing 12.

We can see a basic example of a particularly relevant construct in template metaprogramming, which enables the above-mentioned functionality. There are various interpretations of this construct, depending on its application; in the most direct case, it’s simply a generic struct that exposes its template arguments as statically bound attributes. We can observe an analogous variant of the concept of a generic object in the implementation of traits, where we utilize attributes to define shared behavior for a set of implemented types, much like instantiating a generic type. Traits or interfaces in most languages can then be value-level abstractions, as they provide an abstract or shared view of the behavior exhibited by some set of types. Moving to a higher level of abstraction, we then come to the idea of type traits, which are traits that, as opposed to describing shared behavior, describe a type itself. In the context of C++, this is applied heavily in conjunction with constraints. If we have a means of expressing specific properties of types, it’s not too far-fetched to then use those properties to define compile-time constraint predicates. Stated differently, constraints in the context of C++ are type traits that utilize partial specialization to implement compile-time control flow, which in turn enables compile-time predicates on types. The more broad terminology of this idea is a template metafunction, intending to express a function that operations in terms of template arguments, or just types, as opposed to values and ‘returns’ a type or computation applied to this type. For

completeness, we should also address the fact that the example 12 has a non-type parameter. C++ allows for the passing of constant value-level parameters to templates, which we can use to define compile-time constants that are applied during the template instantiation process. A more formal way of viewing this is that template metafunctions can be defined as dependent types.

```
src/cpp/example/mfuncex.cpp
```

```
1 template <typename T, const char* name>
2 struct MetaInfo {
3     using type = T;
4     static constexpr const char* name = name;
5 };
```

Listing 12: Template metafunction function

4.2.3 Jluna’s Internals. In the most abstract basic sense, we can think of boxing and unboxing to express the idea of a cross-language assignment. The general goal with binding generators, at least those that focus on very idiomatic language usage, is to provide enough abstractions such that working with two languages feels as if we are working with one. More concretely, the goal is to mitigate, as much as possible within certain constraints, the manual specification of bindings and type marshaling code for handling incompatibilities. Hence, if we go back to our basic idea, what we would then strive for would be something in the following form.

```
example/assignmentgoal.cpp
```

```
1 MyType my_type_object = foreign_function();
```

Listing 13: Cross language assignment goal

If we then compare this to the best analog in Jluna, we have this.

```
example/jlunaproxyassign.cpp
```

```
1 RGBA cpp_rgba = Main.safe_eval("return RGBA()");
```

Listing 14: Jluna proxy assignment

Here, ‘Main’ represents the module scope, and ‘safe_eval’ is a wrapper around the Julia C-API function, which uses the runtime to evaluate a snippet of code and return anything specified in a low-level representation. As might already be evident from the code itself, an implicit transformation is occurring. We should clarify that naturally, for this to work, we need existing mirrored-type representation. As we established before, Jluna works quite explicitly by delineating its framework into specification and logic, the specification being the type lens. Furthermore, we know that Julia itself has a variant of lenses, or more simply speaking, it has accessor methods for objects. The main question that arises here what is the implicit transformation happening? The most apparent answer is, well, clear: it’s unboxing. If we have an implicit transformation for some low-level representation, then the assignment must be unboxing. While this is accurate, it doesn’t address the relevant,

more nuanced point of what unboxing and, for that matter, boxing is.

We can think of it on an intuitive level by using the analogy of types of types as recipes. In this context, we can think of a recipe as having three important parts: the ingredients, the instructions, and the tools. The analogy here is that ingredients represent the abstract properties of our constrained type, while the tools represent how we handle these ingredients. This leaves us with instructions that explain how we use the tools in conjunction with the ingredients to obtain our final product, namely the desired type. Since we have the tools and ingredients, we then know that the instructions must represent the logic of how we use these tools to construct a type. Unboxing and boxing then reduce to the question of, for a language A , given the ingredients (our type representation) and the tools (data accessors), how do we use these to construct the type in language B and vice versa.

Some important takeaways from this are that a type representation, regardless of any explicit specification, will ultimately always be a function of the tools, ingredients, and instructions; i.e., the complexity of the type we can construct in the corresponding language only aligns with the specification insofar as it reflects their fundamental overlap. Secondly, there is a dependency between tools, instructions, and generality of constructions. The more a tool is centered on a specific means of construction, the less it depends on instructions. However, this comes at the cost of losing generality, as the tool is inherently less flexible in its application. Finally, the instructions themselves are a function of the tools and ingredients. In the context of binding generators, this means that the logic of how we construct a type is a function of the data accessors and the type representation.

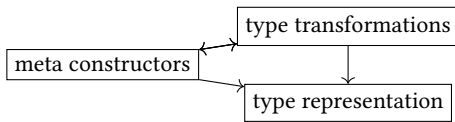


Figure 5: Basic dependencies of the binding generator

In 5, ‘meta constructors’ refer to type construction tools (e.g., lenses), and ‘type transformations’ in this context are simply boxing and unboxing. What follows from this is that if we do want to extend a type of representation, which we do in our case, we must consider a few points.

- (1) *Ingredients:* We have to clearly define what the extended ingredients are for both languages, i.e., what are the new properties of the type representation.
- (2) *Tools:* We must ensure that our tools are sufficient for working with the new ingredients, i.e., do we have the necessary data accessors?
- (3) *Instructions:* We have to define additional logic for how to use the tools in conjunction with the ingredients to construct the new type.

With this, we have sufficient knowledge to understand how the original implementation follows from the ideas introduced. We can consider two abstract forms of types: fundamental and grouped. A record represents a grouped type comprising fields that contain

either fundamental types or other grouped types. As we cannot trivially bridge a record (as it has unbridged members), we must first bridge the fundamental types before the record. To bridge any type, there needs to exist a context in which we have access to our meta constructors, boxing and unboxing logic, and type representations. Furthermore, we need some way to access this context. We can see the approach taken by the author below.

jluna/usertype.inl

```

1  (...) {
2      _mapping.insert({Symbol(name), {
3          [get](T& t_inst) -> unsafe::Value* {
4              return jluna::box<Field_t>(get(t_inst));
5          },
6          [set](T& t_inst, unsafe::Value* val) -> void {
7              set(t_inst, jluna::unbox<Field_t>(val));
8          },
9          Type((jl_datatype_t*) jl_eval_string(
10             as_julia_type<Field_t>::type_name.c_str())
11         )
12     });
13 }
  
```

Listing 15: Add property function

In Listing 15, we can see how the author relates the type specification to a context in which it can be applied to perform bidirectional mappings of this type. The dictionary ‘_mapping’ associates the field name ‘symbol’ (wrapped by a Symbol class) with two lambdas which capture the meta constructors (getters and setters) within their context through C++ lambda capture notation ‘[captured](...args) -> ret_ty { }’, and a wrapped type representation. In this case, we can see that when ‘Field_t’ is a fundamental type, then the box call will be to a simple boxing method, e.g., for primitive conversions. When ‘Field_t’ is a record type, i.e., it has itself more fields (assuming it has a UserType instantiation), we call its box method, which in turn will call the box methods on its field. For completeness, we present an adapted pseudocode version of the box and unbox approach of the original implementation in 16.

The marshaling code in 16 provides a simplified representation of the box and unbox processes. We iterate over the fields specified in the mapping, and for each field, we apply our bridging logic using the meta constructors within the map. These constructors will either bridge the type directly by invoking the fundamental bridging code or, as previously mentioned, call the corresponding bridge method for the record type associated with a specific field, and so on.

A core attribute of the implementation is that it fundamentally relies on nested records existing as fully walkable trees; in other words, the bridging functions must always be supplied with concrete types such that there exists a path over an object that can be fully traced. Hence, this implementation does not account for the case where we encounter an abstract type. The problematic nature, then, is that our implementation of the BVH tree is fundamentally based on abstraction for its representation, or more concretely, as stated before, we have polymorphic nodes represented by an

```

jluna/usertype.inl

1  template<typename A>
2  A Usertype<A>::bridge(B* in) {
3      auto out = A();
4
5      for (auto& pair : _mapping)
6          std::get<bridge_tool>(pair.tools)(out,
7              jluna::safe_call(
8                  B_get("accessor"), in,
9                  (B*) pair.symbol
10             )
11         );
12
13     return out;
14 }

```

Listing 16: Bridge implementation

abstract class. The aim of the extension is, first and foremost, to determine how we can instruct this process on what to do if it encounters an abstract type.

4.3 Implemented Extension

Let's start by creating a high-level breakdown of the user-facing and internal flow of the library, as shown in 6. We could linearly tackle this problem, although it may become somewhat problematic since we might define a specification that is misaligned with the actual type mirroring process. The issue here stems from the different dependencies in the binding generator, as shown in 5.

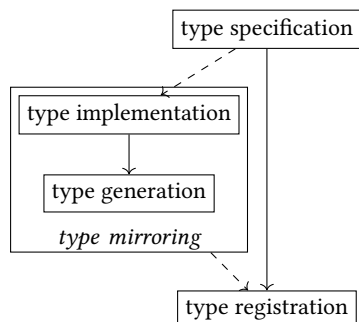


Figure 6: Stages of Jluna's type binding generation process; $\dashrightarrow$ represents user-facing flow, i.e., how the user interacts with the library; $\rightarrow$ represents the internal flow of the library

4.3.1 Representing Fields & Types. We can approach things in a top-down fashion, starting from the fundamental type representation and progressing to the actual bridging code. To bring us back to where we started with our earlier developed formalisms, we had the most basic form of our record represented as a type family indexed by a set of labels. We had three main components to this: the labels, the object type, and the dependent field type. The corresponding implementation in C++ can be seen in Listing 17.

```

src/cpp/usertype.cpp

1  template <
2      ct_string Name,
3      typename T, typename M,
4      M T::* Member
5  >
6  struct Lens<Name, Member> {
7      using type = internal_t<M>;
8
9      static M T::* member_ = Member;
10
11     static auto label() {
12         return Name;
13     }
14     static auto get(T& obj) -> M {
15         return obj.*member_;
16     }
17     template <typename U>
18     static auto set(T& obj, U&& value) -> void {
19         obj.*member_ = value;
20     }
21 };

```

Listing 17: (Template) Field Type meta-function

As a reminder, unlike a typical function, a template metafunction takes a set of statically bound types as its arguments and returns a type as its result. In our case, we have two main components: the name of the field and the *pointer-to-member* type. The name is associated with the Lens using a `ct_string` type, which is a struct that holds a compile-time string and provides added modularity when declaring lenses, as opposed to explicitly declaring static strings. The pointer-to-member type is a commonly used C++ construct that enables us to reference a member of a class or struct. A pointer-to-member type is defined as '`M T::* name`', where '`M`' is the type of the member, '`T`' is the type of the class or struct, and '`name`' is the name of the member. Internally, we expose a stripped type type, which represents the parameterized type variable in case the member is defined as being specialized over a type. To fulfill the requirements of our Lens, we also implement the generic getter and setter methods, in addition to a label reflection mechanism to access the field name from a static context. A notable point of this Lens is that with the setter method, we introduce an additional type variable `U` to account for the case of assigning a derived pointer to a base-type member.

To contrast this with the original implementation here, instead of binding the field properties to the context of the `add_property` function, we isolate them as an entirely static metafunction. There are several benefits to this approach, but the primary motivation is to create a clear separation and generalization between how we specify a type, the tools used to construct it, and the actual logic that applies these tools. Another benefit of this is that, while currently, there exists no native reflection within C++, it should be a relatively non-trivial task to generate these metafunctions at compile time once reflection is introduced in C++26.

A whole type, or more concretely, in our case, object, is then a composition of these fields. As metafunctions typically act on the type level rather than the value level, we pass them to functions as template arguments. Here, we have variadic templates come into play, which allows us to pass an arbitrary number of our metafunctions to a function, in turn allowing us to construct this composition of fields. The original implementation divided the process of defining types into two parts: the C++-sided definition, i.e., type registration through the `add_property` function, and the Julia-sided definition, i.e., type implementation and mirroring. As we wanted to generalize the field or property definition as a function that registers a type over a group of fields as opposed to for each field individually, we adapted the logic into the `init_type` function, the signature of which can be seen in listing 18.

```
libs/jluna/include/usertype.hpp

1  template <typename... FLens, typename... DT>
2  static void init_type(
3      TL<FLens...>,
4      TL<DT...> = TL<>()
5  );
6
7  template <typename... FLens, typename... DT>
8  static void init_dmap(TL<FLens...>, TL<DT...>);
```

Listing 18: Type initialization signature

One caveat we encounter when working with variadic functions is how to delineate between distinct sets of arguments. In our case, we needed not only to pass the set of fields as P but also to pass the derived types relevant for a particular field to perform the necessary concrete cast. To separate the two sets of arguments, we define a variadic container as being a metafunction that acts as a means of enclosing a set of types. Here, we pass these lists as value-level metafunctions, which then allows us to define two distinct sets of type lists `FLens` and `DT`, with `DT` representing our derived types. At compile time, this serves as a means to specialize our type registration for the given specification. At runtime, this specialization is then used to register all the types for the C++-sided code.

The Julia-sided type mirroring happens in C++ through the implementation declaration. We can see the signature of this in the listing 19. The most relevant parts here are the two type parameters T referring to the type of the object and U being the supertype. On a high level, we can think of this as expressing to Julia that within the context of a module (the default just being the global scope), we want to establish the relationship $T <: U$. The type generation triggered by this call is then executed at runtime and calls into the Julia-side portion of the library actually to create a type with this relationship. To avoid redefinition errors, check if a type has already been defined on the Julia side within the specified module scope and reuse existing definitions if they match our C++-sided specification.

How we then use both this C++-sided initialization and its corresponding runtime registration, as well as the Julia-sided mirroring, is shown in an example with listing 20. Here, `TL` represents our basic type list, which contains the properties of the sphere type; the

```
libs/jluna/include/usertype.hpp

1  template<typename T> // class
2  template<typename U> // supertype
3  void Usertype<T>::implement(unsafe::Module* module)
```

Listing 19: Type implementation signature

second argument is the relevant derived types for these properties, in this instance, the Lambertian and Metal materials. The implementation then expresses the relationship `Sphere <: Hittable` and, at runtime, uses the information from the registration to mirror the type with the given properties.

```
src/cpp/usertype.cpp

1  Usertype<Sphere>::initialize_type(
2      lenses<
3          Lens<"center"_ct, &Sphere::center>,
4          Lens<"radius"_ct, &Sphere::radius>,
5          Lens<"r_squared"_ct, &Sphere::r_squared>,
6          Lens<"mat"_ct, &Sphere::mat>,
7          Lens<"bbox"_ct, &Sphere::bbox>
8      >(),
9      types<Lambertian, Metal>()
10 );
11
12 Usertype<Sphere>::implement<Hittable>();
```

Listing 20: User type example application

4.3.2 Semantics of Unboxing & Boxing. Starting with the unboxing process, we can remind ourselves that the core issue here was that the unboxing mechanism we wanted to use for a field of an abstract type had to correspond to the underlying concrete type, given the library’s general design and resulting constraints. The key aspect of the solution in the unboxing case is that, since Julia uses parametric structural subtyping, we typically know the concrete type from the Julia side by simply querying the type info attribute of the low-level object representation used within the C++-sided Jluna code. From this principle, we can construct the concrete unboxing mechanism within Jluna by establishing a correspondence between a pair consisting of a field label and a valid subtype of this field and the unboxing mechanism, i.e., the setter function for this field and its subtype.

```
libs/jluna/include/usertype.hpp

1  static inline tsl::robin_map<
2      DispatchKey, setter_fn<T>,
3      DispatchKeyHash
4  > ty_dispatch_map = {};
```

Listing 21: Type dispatch map

The resulting unboxing mechanism is essentially a simple lookup into this map. The actual instantiation of this map then has two

main cases: fields with concrete types and fields with abstract types. If a field has a concrete type, we create a trivial mapping from the concrete type to the concrete setter. If the field has an abstract type, we resolve all valid derived types and then create a set of mappings to all valid derived types. In simple terms, we create a map consisting of all valid concrete instantiations for all fields of an object. We can define this by cases as follows.

Definition 4.1 (Dispatch Map Entries).

$$\text{entry}(l : T_l) = \begin{cases} \langle l, T_l \rangle & \text{if } \text{isConcrete}(T_l) \\ \prod_{i:I} \langle l, R_i \rangle & \text{for all } (R_i)_{i:I} <: T_l \end{cases}$$

The entry of a dispatch map for a record of type T , a label l and a member type T_l is defined as the projection $\langle l, T_l \rangle \rightarrow \text{set}\langle T_l \rangle(x)$ when T_l is concrete, and the projection $\prod_{i:I} \langle l, R_i \rangle \rightarrow \text{set}\langle R_i \rangle(x)$ when T_l is abstract and R_i are the valid derived types of T_l .

One notable feature that plays a role in dispatch map instantiation is the type filtering mechanism, the pseudo-code of which is shown in Listing 22. An early design choice was that in the type specification, all derived types relevant to a particular object would be passed as the aforementioned template pack, a benefit of this being that with manual specification, we avoid forcing repeated specifications, so if two fields, for example, have the same abstract type we only need to specify the associated derived types once. Furthermore, we reduce implementation complexity somewhat as the set of types does not exist as a member of a Lens but instead as just an auxiliary pack. A key downside to this approach is that we naturally lack a direct reference to the fields that Lens has and the relevant derived types. Thus, when iterating over the fields, we must first filter out derived types that are irrelevant to a specific field to avoid instantiating invalid setters. In simple terms, if a field, for example, has a concrete type A , then we don't want to instantiate setters for any derived types irrelevant to A .

```

libs/jluna/.src/usertype.inl
1 for_each<FilterType<
2     FieldType, is_base_of, DerivedType...
3 >>>();

```

Listing 22: Type filter

The mechanism is generally straightforward. It consists of an initial type T , a binary type predicate $P : \langle A, B \rangle \rightarrow \text{Bool}$ and a set of types T , the resulting filtered type set T_f is then the set of types T for which the predicate $P(T_1, T_2)$ is true. There are various points to be made regarding the pros and cons of this approach. The primary motivation in this case was to simplify the type specification and delegate the actual filtering process to the compiler. Let's consider the automated case of parsing types. This mechanism might have less utility as we could implicitly construct the valid type specification as a part of the parsing algorithm itself. However, it still increases the complexity of the implementation somewhat, regardless, as an implicit valid specification would mean the implementation does not act linearly but instead on a tree of types from the outset.

Algorithm Unboxing usertype

```

1 foreach_indexed<FLens...>([&<Lens FL, size_t I> {
2     invoke_with<FType<FL>>(
3         match {
4             [&<is_jlbits FT>(Value* v) {
5                 FL::set(out, as_pod<FT>(v));
6             },
7             [&<typename FT>(Value* v) {
8                 dispatch_tptr<FT>(out, v);
9             }
10        },
11        nth_field<FL, I>(in /* Value* in */)
12    );
13 });

```

Listing 23: Pseudocode snippet of the unboxing process

Two important optimizations we use to accelerate the unboxing process are immediately setting *piece of data* (PoD) fields and using a *polymorphic inline cache* (PIC) in conjunction with the dispatch map. The first optimization leverages the fact that if a type we receive from Julia has a memory layout that is compatible with the corresponding C++ type, then we can reinterpret the memory payload of the Julia object as the C++ type and set the field directly. This is particularly useful in our case for static arrays as the memory layout of a Julia-sided 'SVectorFloat64, 3' is identical to a C++-sided 'std::array<double, 3>'. The second optimization is the use of a PIC, a common optimization technique in dynamic languages that speeds up method dispatch. In our case, we use the PIC to cache the results of the setter function lookups in the dispatch map. This means that if we have already looked up the setter function for a particular field and concrete type, we can reuse the cached result, which is especially useful for frequent, similar lookups. The size we chose for our PIC is four entries before we resort back to a regular lookup in the dispatch map and evict entries. Further testing is required to evaluate the broader implications of this choice. However, it falls within the original empirically determined [42] and commonly implemented range of around 4-10 entries (e.g., 4 for V8 [36], 6 for SpiderMonkey [55]).

Similarly, in the case of unboxing, the boxing process requires that we know the underlying concrete type so we can use the correct mechanism to map to the Julia-side type. We utilize the exact type pattern matching applied in the dispatch map construction to differentiate between boxing concrete and abstract types. The concrete case is a direct call to the appropriate boxing function, essentially unchanged from the original implementation. The abstract case requires us to determine the correct concrete type. Unlike in the case of Julia, C++ provides no clean, native way of querying the concrete type of an abstract pointer. The most trivial and safe solution, albeit at the cost of performance, is to brute-force test all valid derived types using a dynamic cast, essentially treating the abstract type as a union. As the library is designed to require no additional modifications to types themselves, we cannot expect

any self-tagging mechanisms to be present. Hence, any scheme for determining the concrete type must use only the information provided by the type specification and the resulting runtime types we are passed.

Algorithm Boxing usertype fields

```

1  invoke(
2    match {
3      [&]<concrete FT>(FT& v) {
4        return box<FT>(v);
5      },
6      [&]<abstract FT>(FT& v) {
7        return dispatch_vptr(v);
8      }
9    },
10   FLens::get(in /* T& in */)
11 )...
```

Listing 24: Pseudocode snippet of the boxing process

The fast, albeit dangerous, solution we ended up implementing to avoid brute force dynamic casts was to create a dispatch map based on the vptr of the derived types. The fact we can exploit's that, despite there being no strict specification of how vptrs are implemented, the convention most compilers end up adopting is having the vptr as a hidden 0th member of the class. Furthermore, since vptrs definitionally act as unique tags to a type-specific table of virtual functions, we can leverage this to create a mapping of vptrs to unbox functions of the corresponding type. In essence, we are creating an auxiliary extension to the vtables of all derived types using the vptrs as a key to the correct boxing function. One future optimization is to make our PIC bidirectional by generalizing it to support both boxing and unboxing.

4.4 Evaluation

Our evaluation will consist of two main parts: the first is a performance evaluation of the library, and the second is a qualitative evaluation of the developer and user experience. The performance evaluation will detail the benchmarking setup, the performance of boxing and unboxing, and the overhead introduced by the application of JuliaCall and Jlna.

4.4.1 Benchmarking & Performance. On a high level, both JuliaCall and Jlna function on similar principles for establishing a line of communication between C++ and Julia. Both libraries provide us with a means to start the Julia runtime, call Julia-sided functions, or conversely define C++ or Python-sided functions that can be called from Julia. In our applied instance, we want to intercept the call to the BVH traversal function and then call into our foreign functions from Julia. The approach we take to intercept the Julia-sided call is to redefine the traversal function within the scope of the Julia ray-tracing module, replacing the original function with one that calls into our foreign code. The internal trampoline used by the redefined function is created using the respective interop libraries,

both of which, in essence, more or less generate wrappers around Julia's 'ccall' function.

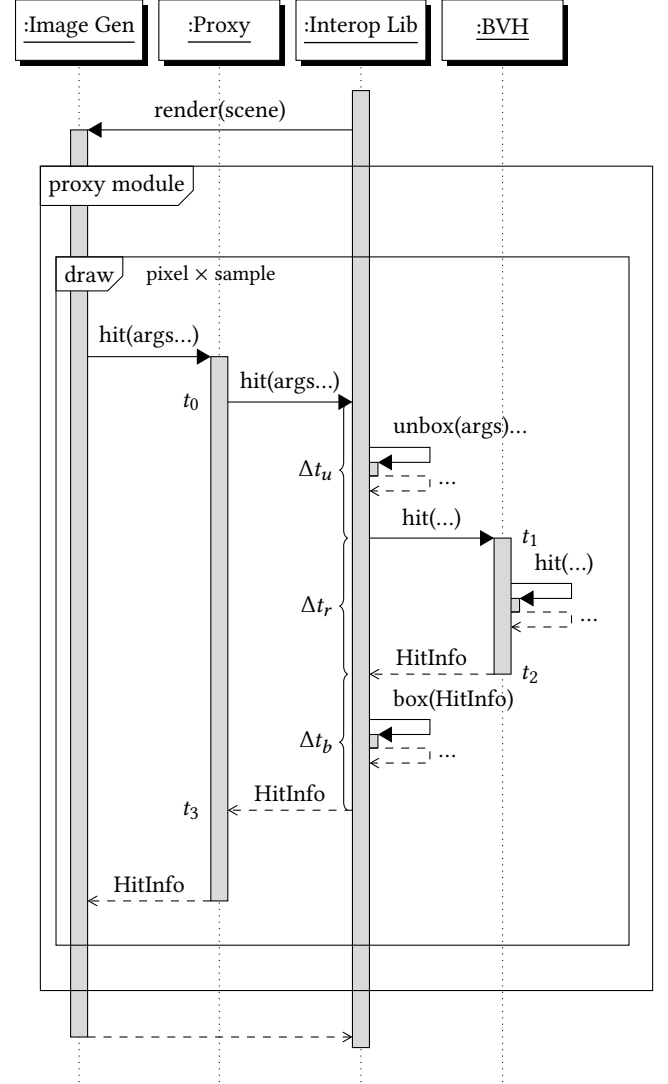


Figure 7: Benchmarking setup

We have created a sequence diagram for the benchmarking setup in 7. The first timing point is recorded right before calling the generated ccall trampoline and is denoted by t_0 . This calls into the respective interop libraries, which proxy the user-defined code in our foreign language, i.e., our rewritten BVH traversals. The second timing point, t_1 , is recorded once we have entered the user-defined traversal functions, marking the start of the actual work. Similarly, we generate a third timing point, t_2 , prior to boxing once our traversal is completed and a fourth timing point, t_3 , after the boxing has concluded and our value is returned to the Julia runtime. Additionally, we also recorded the time spent within Julia's GC for the entire duration between t_0 and t_3 .

All benchmark data are derived from the task of rendering the same cottage scene using the C++ and Python implementations of the BVH traversal. Naturally, as our focus is on the performance of

the libraries rather than the performance of the actual ray-tracing algorithms, we primarily discuss Δt_u and Δt_b , the time spent on unboxing and boxing, respectively. The actual work time, Δt_r , does have relevance, though, insofar as it relates to the object representations derived from the unboxing process.

A simple overview of the results can be seen in figure 8. The most apparent observation we can make from this is that boxing and unboxing times are considerably higher in C++ than in Python. Taking unboxing as an example, within Jluna, we average around 75.8 μs per unboxing, regardless of all arguments, whereas in Python, we only average around 5.8 μs per unboxing. The boxing times are similarly skewed, with C++ averaging around 59.3 μs per boxing and Python averaging around 3.1 μs per boxing. We can attribute this difference in large part to the nature of the underlying implementations, namely copying vs. non-copying interoperability.

JuliaCall, like other Python interop libraries, only builds wrapper objects, choosing not to copy the actual data. Instead, it relies on proxying or intercepting getter and setter operations using the Julia C-API. In practice, this means that the only data we create when “unboxing” in JuliaCall are minimal proxy objects, which more or less wrap a Julia-sided pointer. In contrast, Jluna (in the way we apply it) copies over the actual data, as we intended to leave our original C++ implementation unchanged. This means that our unboxing process in Jluna fundamentally necessitated a de facto deep copy of the data, in turn leading to higher unboxing and boxing times.

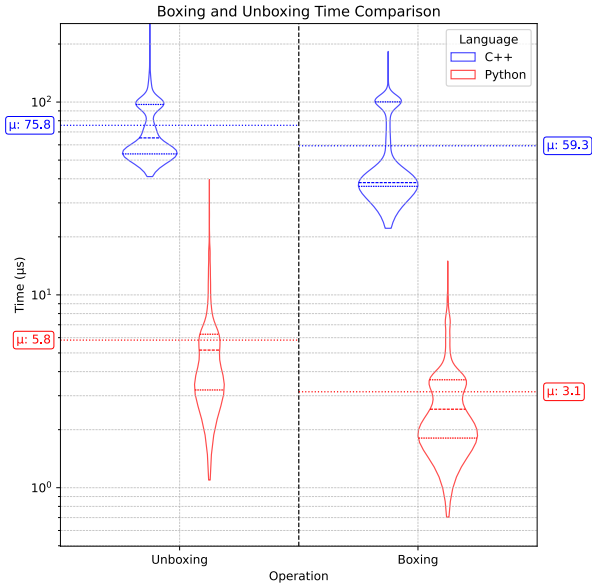


Figure 8: Boxing and unboxing comparisons

We can observe a more granular breakdown of the boxing and unboxing times in figure 9. An interesting observation from this figure is especially the structural differences in the execution time of different marshaling operations for the ray casts. Within Python, we observe consistent spikes in both boxing and unboxing times, particularly between the 400th and 800th ray casts. It’s difficult to say precisely what the source of these spikes is, but a potential explanation could be the Python GC. The C++ implementation

behaves considerably more inconsistently; the most striking observation is the sudden high-frequency spikes in the boxing times. We can notice similar, albeit less pronounced, spikes in the unboxing times. When we consider the otherwise relatively flat profile of Boxing and unboxing times in C++, we could attribute inconsistent cache locality as a potential source of these spikes.

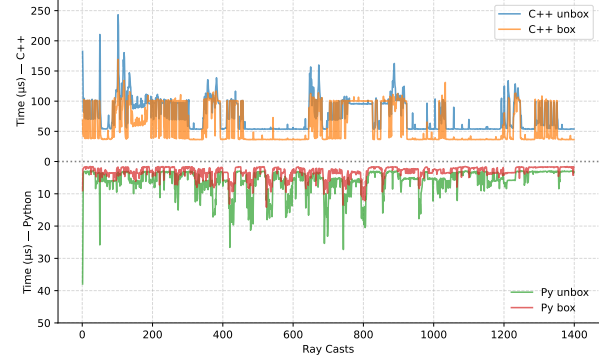


Figure 9: Boxing and unboxing mirrored

One observation made while executing the Python implementation was that there were occasional stutters in the execution. However, this appeared to contradict the otherwise relatively consistent performance noted in both 8 and 9. To investigate this further, we looked at specifically the time spent in the Julia GC within the Python implementation. It’s clear, looking at 9, that these stutters don’t appear to occur in the boxing or unboxing times; hence, they must be occurring during the execution of the actual work within Python. To investigate this more clearly, we graphed the total overhead of the Python marshaling process, along with the time spent in the Julia GC for each ray cast in figure 10.

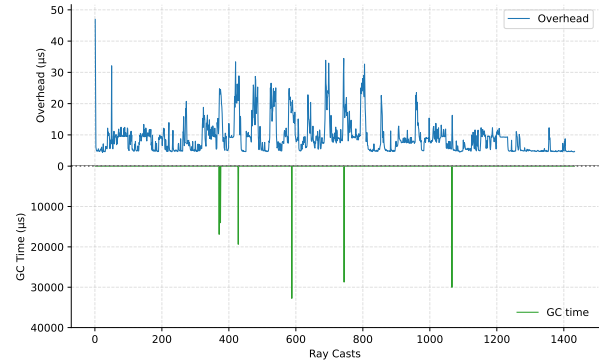


Figure 10: Overhead mirrored

Something that stands out in 10 is just how significantly larger the GC spikes are during the execution of the Python BVH traversal than the actual overhead of the marshaling process. In other words, the overhead of the marshaling process in Python is minimal, with no recorded time spent within the Julia GC. However, the actual application of the wrapper types within our Python BVH traversal contributes significantly to the overall runtime. We can emphasize this more clearly by looking at the cumulative overhead in figure 11.

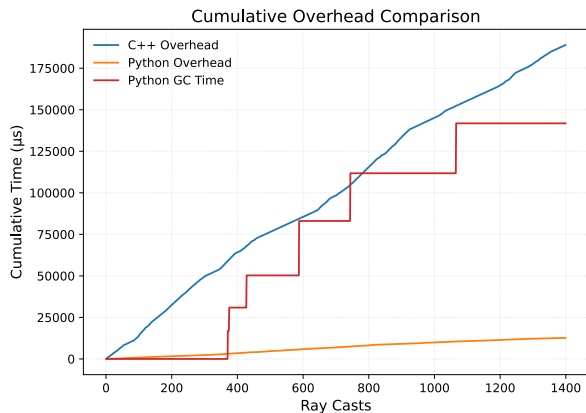


Figure 11: Cumulative overhead

In 11 we can see that the marshaling overhead shows us the same profile we observed in 8, but the overhead introduced by the time spent in the GC for the BVH traversal of the Python implementation puts us at times almost on the same level as the overhead introduced by just copying the actual data as opposed to holding the pointers and proxying all operations.

Metric	Python		C++	
	Mean (μs)	Σ (μs)	Mean (μs)	Σ (μs)
Unboxing	5.8284	8 346.32	75.7565	105 983.308
Boxing	3.1438	4 501.9580	59.3206	82 989.486
Actual	1 874.7250	2 684 606.196	0.1723	241.0350
Overhead	8.9723	12 848.278	135.0771	188 972.794

Table 2: Comparison of average and total times for unboxing, boxing, actual work, and overhead in Python vs. C++.

Something we can take away from the performance evaluation is that while the C++ implementation does have a higher overhead in terms of boxing and unboxing, as seen again in 2, it does not suffer from the same GC overhead as the Python implementation. This is primarily because we copy over the actual data in C++. As the Julia GC only executes within the context of the Julia runtime, it’s clear that in this instance, a drawback of repeatedly calling into the Julia runtime, as is the case with JuliaCall, is that we incur a significant overhead in terms of the time spent in the GC. In contrast, Jlna has no recorded time spent in the Julia GC, as the user-defined code is executed entirely within C++, and the marshaling process is implemented to minimize GC involvement.

4.4.2 Developer & User Experience. One aspect the original implementation had in mind, which we tried to continue with our extension, was utilizing modern C++ features to improve the developer experience of working with the code. The main aspects of this were the use of well-documented and type-safe template metaprogramming, in addition to functional programming and, more broadly, declarative programming paradigms. In practice, this meant we aimed to design the implementation—especially the more abstract portions of the code—so that they would reflect the most critical parts of the implementation with as little noise as possible.

As templates are somewhat notoriously complex to read, we followed the style of the original implementation by keeping heavy template or type predicate code separate from the main implementation, and we ensured that types have clear and descriptive names. Functional or more complex compile-time type constructs were integrated in a way that reflects a sort of two-level language, where abstractions of templates are used in a manner more naturally akin to a functional compile-time programming language, separate from the runtime implementation, and with the underlying type logic being separated from the actual implementation.

The user experience is multifaceted, depending on the user’s priorities. While the library does not yet achieve fully implicit interoperability, it does provide a relatively simple interface for declaring types and functions. In its current state, this is a mundane and time-consuming action, depending on the size of the codebase, but it could be automated with relative ease once reflection is introduced in C++26. A key benefit of this library is that, despite requiring some explicit declarations, these are largely separate from the main implementation. This isolation allows for more versatility in the application of the library, as opposed to those that require a tighter coupling between the interoperability code and the main implementations in either language. As mentioned earlier, performance overhead is a factor that requires careful consideration when using the library. To aid in this, a basic user-facing benchmarking tool was implemented for the C++ side, and Julia, being a dynamic high-level language, has various tools to benchmark from its end.

4.5 Limitations & Threats to Validity

We can understand the limitations of this extension and, more broadly, the idea of expanding interoperability libraries to support more complex types by first assessing some important language-level considerations and then examining the broader landscape of the evolution of programming languages and their real-world applications.

4.5.1 Slow and varying generics support. Generics support, or more complex type-level programming, is a key aspect of building more implicit, idiomatic interoperability libraries that aren’t reliant on external scripts or tools to parse and generate the necessary code. C++ and Julia are both languages that have, in their ways, developed considerably complex type systems. A benefit of this focus in their design is that we can leverage these systems, especially on the C++-sided end, to approach the problem of handling many different types in highly generic ways, thereby minimizing the amount of boilerplate code we need to write. Treating the problem of interoperability with this type-level approach, while quite interesting in this case, could pose a significantly larger challenge in languages that don’t have as developed type systems. Furthermore, it’s not even a guarantee that languages will continue to develop increasingly complex type systems, as even general-purpose programming languages often end up having domain-specific requirements or design choices that can deliberately limit a highly expressive type system. A type-level approach is a suitable approach in languages that easily support it, but it could be detrimental in languages based on incompatible design philosophies.

4.5.2 Asymmetric metaprogramming features. As briefly touched upon in the previous section, due to the varying development and design of different programming languages, they often have very different metaprogramming features. As our implementation is heavily reliant on metaprogramming features for things like type-level functional logic, reflection, type generation, specialization, and so on, similarly to the previous point, it may not be easy to use these design choices as a more general framework for interoperability systems. Metaprogramming exists to various degrees in many languages. However, its feature set should be closely evaluated to determine if it's sufficient to use as the basis for an interoperability library or if other approaches would be more suitable.

4.5.3 Decentralized language evolution. If we examine the broader landscape of programming languages, we can observe a general trend: languages tend to evolve within their ecosystems. Naturally, there are exceptions to this trend, and various attempts have been made to build languages and more abstract frameworks that enable different languages to coexist within the same ecosystem. A key example of this is the JVM, CLR, and, more recently, the GraalVM. However, despite these developments, many mainstream popular languages continue to evolve within their respective communities and ecosystems. This is not to say that these languages are entirely isolated from one another. We do often see theoretical or practical aspects of their implementation being inspired by other languages or built on existing tools, a prime example of this being the set of languages developed on LLVM. Even with the often theoretical or technological overlap of languages, a limiting factor in creating traditional interop libraries that enable the bridging of more complex properties of languages, especially ones that diverge in their design philosophies, is that the languages are often inherently not designed to interoperate as we often see them created and evolve to address either a deficit in another language or some completely different domain-specific niche. We have seen various developments aimed at bridging separate language ecosystems, such as WASM or the C-ABI. However, these are often limited, and as we established, they come with certain fundamental drawbacks. We have also observed that languages built on an inherently unifying framework can overcome many of the deficits of more traditional interop libraries. However, existing, well-established language ecosystems continue to be the primary means by which people interact and work with these languages.

4.5.4 Theoretical exploration vs practical needs. A final point to consider is the fundamental reality or question of "is interoperability past basic constructs even necessary in the first place?". In an industry setting, especially at larger scales, it's somewhat natural for the environment to focus on the more short-term to mid-term outcomes of a piece of software. This tends to lead to a preference for ease of use, safety, reliability, and other factors. An observation that can be made from this preference for safe, reliable, and easy software in a practical setting is that it often rarely attempts to evolve beyond what is actually needed and what can be reliably trusted. Legacy systems and codebases often result from this preference. While this is not inherently a bad thing, it does, in many instances, mean that, rather than spending much time exploring or ruminating on the latest features in a language, the focus is more on simply working with what we have to get the job done. Exploration and being more

open to new technologies tend to be more common in a research setting or sometimes only possible in instances where something is financially viable. Creating more expressive interoperability, while interesting to explore in a theoretical sense, is not necessarily a real practical need for many developers. Research and academia being ahead of what is used in day-to-day life is not a new phenomenon; however, in this instance, there is a clear need for more research to assess whether this approach to interoperability is more broadly applicable, given the constraints elaborated on above.

5 CONCLUSION

Understanding the modern state of language interoperability is a complex task. Everything, from the lowest level of the programming stack to the highest level of abstraction, plays a role in how well languages interact with one another. Despite this complexity, a structure exists that can be understood and further utilized to comprehend better how interoperability tools have evolved and are likely to continue evolving. The goal of this paper was to develop a framework for understanding the evolution of more traditionally based interoperability tools and to build upon this framework, along with an existing tool, to explore more complex, niche, and potential future applications and assess their broader viability. The primary insight of this paper is that FFI-based tools, despite their long history, remain highly relevant and have considerable room for further evolution in the future. A secondary insight is that FFI and more unified execution models, along with their associated interoperability tools, are not entirely mutually exclusive. This means that many of the more recent theoretical foundations of unified execution models can be applied to FFI-based tools to enhance their performance and usability. Nonetheless, it's ultimately challenging to predict how things will evolve in the future; however, it's pretty evident that language interoperability is a complex and multifaceted problem deeply tied to the evolution of programming languages.

REFERENCES

- [1] M. Abadi and L. Cardelli, *A Theory of Objects*. Springer New York, 1996, ISBN: 9781461264453. DOI: [10.1007/978-1-4419-8598-9](https://doi.org/10.1007/978-1-4419-8598-9).
- [2] D. Abrahams and S. Seefeld, *Boost.python library documentation*, Version 1.72.0, Accessed: 2025-06-01, 2019. [Online]. Available: https://www.boost.org/doc/libs/1_72_0/libs/python/doc/html/index.html.
- [3] S. Akbary, *Wasienv: Wasi development workflow for humans*, <https://blog.wasmer.io/wasienv-wasi-development-workflow-for-humans-1811d9a50345>, Accessed: 2025-05-27, 2019.
- [4] S. Akbary, M. McCaskey, J. Kruth, *et al.*, *Wasienv/wasienv*, Jun. 6, 2023. [Online]. Available: <https://github.com/wasienv/wasienv>.
- [5] Armin Rigo and contributors, *Cffi documentation*, C Foreign Function Interface for Python, 2024. [Online]. Available: <https://cffi.readthedocs.io/en/latest/> (visited on May 25, 2025).
- [6] S. Audet and Contributors, *Javacpp - the missing bridge between java and native c++*, GitHub repository, 2024. [Online]. Available: <https://github.com/bytedeco/javacpp>.

- [7] Ayke, R. Evans, D. Gryski, *et al.*, *Tinygo-org/tinygo*, Nov. 14, 2024. [Online]. Available: <https://github.com/tinygo-org/tinygo>.
- [8] E. Barrett, C. F. Bolz, and L. Tratt, “Approaches to interpreter composition”, *Computer Languages Systems and Structures*, vol. 44, pp. 199–217, Dec. 2015, issn: 1477-8424. doi: [10.1016/j.cl.2015.03.001](https://doi.org/10.1016/j.cl.2015.03.001).
- [9] N. G. de Bruijn, “Telescopic mappings in typed lambda calculus”, *Information and Computation*, vol. 91, no. 2, pp. 189–204, 1991. doi: [10.1016/0890-5401\(91\)90066-B](https://doi.org/10.1016/0890-5401(91)90066-B).
- [10] Bytecode Alliance, *Bytecode alliance projects*, Accessed: 2025-05-27, 2023. [Online]. Available: <https://bytecodealliance.org/projects>.
- [11] Bytecode Alliance, *Cargo-wasi: A lightweight cargo subcommand for the wasm32-wasi target*, Provides tools for building, running, testing, and benchmarking Rust code for the wasm32-wasi target. Licensed under Apache-2.0 with LLVM exception., 2024. [Online]. Available: <https://github.com/bytecodealliance/cargo-wasi>.
- [12] Bytecode Alliance, *Wasmtime documentation*, Wasmtime is a standalone runtime for WebAssembly, WASI, and the Component Model. It is designed to be fast, secure, configurable, and standards compliant, and can run WebAssembly outside of the web environment., 2024. [Online]. Available: <https://github.com/bytecodealliance/wasmtime>.
- [13] Bytecode Alliance, *Component model concepts*, <https://component-model.bytecodealliance.org/design/component-model-concepts.html>, Accessed: 2025-05-27, 2025.
- [14] C. Carruth, *Object relocation in terms of move plus destroy*, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p1144r13.html>, ISO/IEC JTC1/SC22/WG21 P1144R13, Apr. 2025.
- [15] Clang Documentation, *Objective-c automatic reference counting (arc)*, Accessed: 2025-06-02, 2023. [Online]. Available: <https://clang.llvm.org/docs/AutomaticReferenceCounting.html>.
- [16] Clang Documentation Contributors, *Libclang - c interface to clang*, <https://clang.llvm.org/docs/LibClang.html>, Accessed: 2025-06-12, 2025.
- [17] Clang Team, *Clang attribute reference: trivial_abi*, <https://clang.llvm.org/docs/AttributeReference.html#trivial-abi>, Accessed: 2025-06-04, 2025.
- [18] B. Clarke, D. Elkins, J. Gibbons, *et al.*, “Profunctor Optics, a Categorical Update”, *Compositionality*, vol. 6, p. 1, 2024, issn: 2631-4444. doi: [10.32408/compositionality-6-1](https://doi.org/10.32408/compositionality-6-1).
- [19] J. contributors, *Garbage collection and javacall*, <https://juliainterop.github.io/JavaCall.jl/gc.html>, Accessed: 2025-06-05.
- [20] R. Cooper, “Records and record types in semantic theory”, in *Journal of Logic and Computation*, vol. 15, no. 2, pp. 99–112, Apr. 1, 2005, issn: 0955-792X. doi: [10.1093/logcom/exi004](https://doi.org/10.1093/logcom/exi004).
- [21] R. Cooper, “So whats all this structure good for? some uses of record types in ttr”, in *Proceedings of the ESSLLI 2021 Workshop on Computing Semantics with Types, Frames and Related Structures*, Association for Computational Linguistics, Utrecht, The Netherlands (online), Jul. 2021, pp. 3–11.
- [22] C. Cords, *Jluna: A modern julia wrapper for c++*, <https://www.clemens-cords.com/jluna>, v1.0.0, 2022. [Online]. Available: <https://www.github.com/clemens-cords/jluna>.
- [23] cppreference.com, *C++ language: Class specifier*, Accessed: 2025-06-02, 2024. [Online]. Available: <https://en.cppreference.com/w/cpp/language/class>.
- [24] Crubit Contributors, *Struct layout*, https://github.com/google/crubit/blob/8d6b38fd9ab044ab7595a7df9752e62eb84345c7/docs/design/struct_layout.md, Accessed: 2025-06-04, 2023.
- [25] T. C. P. Developers, *Crubit design document: Cxx section*, <https://github.com/google/crubit/blob/main/docs/design/design.md#cxx>, Accessed: 2025-05-29, 2025.
- [26] T. J. Developers, *Libcxxwrap-julia: C++ library for backing cxxwrap.jl*, <https://github.com/JuliaInterop/libcxxwrap-julia>, Accessed: 2025-06-01, 2025.
- [27] DigitalOcean Community, *Java JVM memory model - memory management in java*, Accessed: 2025-06-05, 2020. [Online]. Available: <https://www.digitalocean.com/community/tutorials/java-jvm-memory-model-memory-management-in-java%7D>.
- [28] DutChen18, *leaningtech/ts2cpp*, Sep. 2, 2024. [Online]. Available: <https://github.com/leaningtech/ts2cpp>.
- [29] Emscripten Project, *File system overview — emscripten documentation*, Accessed: 2025-05-27, n.d. [Online]. Available: https://emscripten.org/docs/porting/files/file_systems_overview.html#file-system-overview.
- [30] J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, and A. Schmitt, “Combinators for bi-directional tree transformations”, *POPL05: The 32nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages 2005*, ACM, Jan. 12, 2005, pp. 233–246. doi: [10.1145/1040305.1040325](https://doi.org/10.1145/1040305.1040325).
- [31] W. S. Fulton, O. Betts, VZ, *et al.*, *swig/swig*, Feb. 5, 2025. [Online]. Available: <https://github.com/swig/swig>.
- [32] Y. Futamura, “Partial computation of programs”, in *RIMS Symposia on Software Science and Engineering*, Springer Berlin Heidelberg, 1983, pp. 1–35, isbn: 9783540394426. doi: [10.1007/3-540-11980-9_13](https://doi.org/10.1007/3-540-11980-9_13).
- [33] O. Goffart, N. Layzell, twistedfall, *et al.*, *mystor/rust-cpp*, Nov. 20, 2024. [Online]. Available: <https://github.com/mystor/rust-cpp>.
- [34] D. Gohman, A. Crichton, A. Brown, *et al.*, *Webassembly/wasi-sdk*, Oct. 29, 2024. [Online]. Available: <https://github.com/WebAssembly/wasi-sdk>.
- [35] Google, *Crubit: C++/rust bidirectional interop tool*, GitHub repository, 2024. [Online]. Available: <https://github.com/google/crubit>.
- [36] Google, *Wasm-constants.h*, <https://chromium.googlesource.com/v8/v8/+refs/heads/main/src/wasm/wasm-constants.h>, Accessed: 2025-07-06, n.d.
- [37] Google and Contributors, *Cxx - safe interoperability between rust and c++*, Project documentation, 2024. [Online]. Available: <https://cxx.rs/>.
- [38] GraalVM Contributors, *Graalvm languages reference manual*, <https://www.graalvm.org/latest/reference-manual/languages/>, Accessed: 2025-05-27, 2025.

- [39] GraalVM Contributors, *Graalvm webassembly documentation*, <https://www.graalvm.org/webassembly/>, Accessed: 2025-05-27, 2025.
- [40] GraalVM Team, *Introduction to graalvm*, Accessed: 2025-05-25, 2025. [Online]. Available: <https://www.graalvm.org/latest/introduction/>.
- [41] J. Hartzell, *C++ Move Semantics Considered Harmful (Rust is better)*, The Coded Message (blog), Accessed: 2025-07-07, Nov. 3, 2021. [Online]. Available: <https://www.thecodedmessage.com/posts/cpp-move/>.
- [42] U. Hlzl, C. Chambers, and D. Ungar, “Optimizing dynamically-typed object-oriented languages with polymorphic inline caches”, Nov. 1999.
- [43] W. Jakob, H. Schreiner, J. Rhineland, *et al.*, *Pybind/pybind11*, Nov. 14, 2024. [Online]. Available: <https://github.com/pybind/pybind11>.
- [44] B. Janssens, P. Jain, M. Horn, *et al.*, *JuliaInterop/CxxWrap.jl*, Jan. 29, 2025. [Online]. Available: <https://github.com/JuliaInterop/CxxWrap.jl>.
- [45] S. G. Johnson, T. Arakaki, Y. Yu, *et al.*, *JuliaPy/PyCall.jl*, Mar. 11, 2024. [Online]. Available: <https://github.com/JuliaPy/PyCall.jl>.
- [46] JuliaInterop, *Cxx.jl: The julia c++ interface*, <https://github.com/JuliaInterop/Cxx.jl>, Accessed: 2025-06-01, 2025.
- [47] H. Kalbasi, *Zngur: A c++/rust interop tool*, <https://github.com/HKalbasi/zngur>, Accessed: 2025-05-29, 2025.
- [48] H. Kalbasi, *Design decisions*, <https://hkalbasi.github.io/zngur/philosophy.html>, Accessed: 2025-05-31, 2025.
- [49] B. Kundu, V. Vassilev, and W. Lavrijsen, “Efficient and accurate automatic python bindings with cppy; cling”, 2023. doi: [10.48550/ARXIV.2304.02712](https://doi.org/10.48550/ARXIV.2304.02712). arXiv: [2304.02712](https://arxiv.org/abs/2304.02712).
- [50] B. H. Liskov and J. M. Wing, “A behavioral notion of subtyping”, *ACM Trans. Program. Lang. Syst.*, vol. 16, no. 6, pp. 1811–1841, Nov. 1994, issn: 0164-0925. doi: [10.1145/197320.197383](https://doi.org/10.1145/197320.197383).
- [51] J. Maurer, *Trivial relocatability*, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2786r11.html>, ISO/IEC JTC1/SC22/WG21 P2786R11, Nov. 2024.
- [52] Microsoft Corporation, *Common language runtime (clr) overview*, Accessed: 2025-05-25, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/standard/clr>.
- [53] Microsoft Learn, *Object lifetime and resource management (raii)*, Accessed: 2025-06-02, 2022. [Online]. Available: <https://learn.microsoft.com/en-us/cpp/cpp/object-lifetime-and-resource-management-modern-cpp?view=msvc-170>.
- [54] R. Montague, “English as a formal language”, in *Linguaggi nella società e nella tecnica*, B. Visentini, G. Scarpa, and B. Segre, Eds., (Reprinted in Montague, “Formal Philosophy”, Yale University Press, 1974), Edizioni di Comunità, 1970, pp. 188–221.
- [55] Mozilla, *ICState.h*, Source code, [js/src/jit/ICState.h](https://searchfox.org/mozilla-central/source/js/src/jit/ICState.h), Mozilla SpiderMonkey, n.d. [Online]. Available: <https://searchfox.org/mozilla-central/source/js/src/jit/ICState.h> (visited on Jul. 6, 2025).
- [56] nLab authors, *Structure*, <https://ncatlab.org/nlab/show/structure>, Jul. 2025. [Online]. Available: <https://ncatlab.org/nlab/revision/structure/54>.
- [57] Office of the National Cyber Director, “Back to the building blocks: A path toward secure and measurable software”, The White House, Tech. Rep., Feb. 2024, Accessed: 2025-05-29. [Online]. Available: <https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf>.
- [58] S. Pestov, *Library evolution in swift*, <https://swift.org/blog/library-evolution/>, Accessed: 2025-05-27, 2020.
- [59] S. H. Peter Shirley Trevor David Black, *Ray tracing in one weekend*, 2024. [Online]. Available: <https://raytracing.github.io/books/RayTracingInOneWeekend.html>.
- [60] S. H. Peter Shirley Trevor David Black, *Ray tracing: The next week*, 2024. [Online]. Available: <https://raytracing.github.io/books/RayTracingTheNextWeek.html>.
- [61] C. Piovesan, S. De Rossi, Alex, *et al.*, *leaningtech/cheerp-meta*, Mar. 29, 2025. [Online]. Available: <https://github.com/leaningtech/cheerp-meta>.
- [62] M. Riley, “Categories of Optics”, 2018. doi: [10.48550/ARXIV.1809.00738](https://doi.org/10.48550/ARXIV.1809.00738). arXiv: [1809.00738](https://arxiv.org/abs/1809.00738).
- [63] J. Rose, *Abi stability and more*, Accessed: 2025-05-25, Feb. 2019. [Online]. Available: <https://swift.org/blog/abi-stability-and-more/>.
- [64] C. Rowley, *Juliacall: Seamless integration of julia and python*, <https://pypi.org/project/juliacall/>, Version 0.9.25, released on May 14, 2025. Accessed: 2025-05-27, 2025.
- [65] C. Rowley, *github-actions[bot]*, M. Cranmer, *et al.*, *JuliaPy/Python-Call.jl*, Jan. 22, 2025. [Online]. Available: <https://github.com/JuliaPy/PythonCall.jl>.
- [66] Rust Foundation, *Google contributes \$1m to rust foundation to support c++/rust “interop initiative”*, Accessed: 2025-05-29, Feb. 2024. [Online]. Available: <https://rustfoundation.org/media/google-contributes-1m-to-rust-foundation-to-support-c-rust-interop-initiative/7D>.
- [67] A. Sengupta, M. Kittisopikul, B. Burdick, *et al.*, *JuliaInterop/JavaCall.jl*, Nov. 22, 2023. [Online]. Available: <https://github.com/JuliaInterop/JavaCall.jl>.
- [68] SingleStore Labs, *Wasix: Posix compatibility layer for wasi builds*, Provides stubs for POSIX compatibility in WASI builds, facilitating the compilation of sources expecting POSIX support. Designed for experimental use., 2024. [Online]. Available: <https://github.com/singlestore-labs/wasix>.
- [69] A. Taylor, P. Craig, B. Silverman, *et al.*, *google/autocxx*, Mar. 5, 2025. [Online]. Available: <https://github.com/google/autocxx>.
- [70] The Chromium Project Developers, *Rust and c++ interoperability*, <https://www.chromium.org/Home/chromium-security/memory-safety/rust-and-c-interoperability/>, Accessed: 2025-05-29, 2020.
- [71] The Julia Language, *Embedding julia*, Julia 1.x Documentation, 2024. [Online]. Available: <https://docs.julialang.org/en/v1/manual/embedding/> (visited on May 25, 2025).
- [72] The Julia Programming Language, *Just-in-time (jit) compilation — julia documentation*, Accessed: 2025-05-27, 2024. [Online]. Available: <https://docs.julialang.org/en/v1/devdocs/jit/>.

- [73] The Julia Project, *Julia documentation: The julia object model*, <https://docs.julialang.org/en/v1/devdocs/object/>, Accessed: 2025-05-31, 2024.
- [74] The Rust Project Developers, *Foreign function interface (ffi)*, Rustonomicon: Unsafe Rust Guidelines, 2024. [Online]. Available: <https://doc.rust-lang.org/nomicon/ffi.html> (visited on May 25, 2025).
- [75] The Rust Project Developers, *What is pinning?*, <https://doc.rust-lang.org/std/pin/index.html#what-is-pinning>, Accessed: 2025-06-04, 2025.
- [76] The rust-bindgen Developers, *Rust-bindgen*, <https://github.com/rust-lang/rust-bindgen>, Accessed: 2025-05-29, 2025.
- [77] D. Tolnay, *Cxx: Safe interop between rust and c++*, <https://github.com/dtolnay/cxx>, Accessed: 2025-05-29, 2025.
- [78] D. Tolnay, *syn: A parsing library for rust source code*, <https://docs.rs/syn/latest/syn/>, Latest version published on crates.io as of June 2025, 2025. [Online]. Available: <https://docs.rs/syn/latest/syn/>.
- [79] E. Unruh, *Prime number computation*, ANSI X3J16-94-0075 / ISO WG21-462, 1994.
- [80] V. Vassilev, A. Naumann, marsupial, *et al.*, *root-project/cling*. Mar. 7, 2025. [Online]. Available: <https://github.com/root-project/cling>.
- [81] T. L. Veldhuizen, *C++ templates as partial evaluation*, 1998. doi: [10.48550/ARXIV.CS/9810010](https://arxiv.org/abs/10.48550/ARXIV.CS/9810010).
- [82] WASI Subgroup, W3C WebAssembly Community Group, *Wasi.dev: The webassembly system interface*, <https://wasi.dev/>, Accessed: 2025-05-27, 2025.
- [83] WasmEdge Contributors, *Build and run with ahead-of-time compilation*, <https://wasmedge.org/docs/start/build-and-run/aot/>, Accessed: 2025-05-27, 2025.
- [84] WebAssembly Community Group, *Webassembly features*, <https://webassembly.org/features/>, Accessed: 2025-05-27, 2025.
- [85] M. Young, silvanshade, and A. Taylor, *google/moveit*. Jun. 15, 2023. [Online]. Available: <https://github.com/google/moveit>.
- [86] A. Zakai, S. Clegg, juj, *et al.*, *emscripten-core/emscripten*. May 24, 2025. [Online]. Available: <https://github.com/emscripten-core/emscripten>.
- [87] F. Zappa Nardelli, J. Belyakova, A. Pelenitsyn, B. Chung, J. Bezanson, and J. Vitek, “Julia subtyping: A rational reconstruction”, en, *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–27, Oct. 24, 2018, issn: 2475-1421. doi: [10.1145/3276483](https://arxiv.org/abs/10.1145/3276483).
- [88] E. Zhdan, “Swift / C++ Interoperability”, in *EuroLLVM Developers’ Meeting 2024*, Accessed: 2025-06-12, Apr. 2024.
- [89] J. Zucker, “Formalization of classical mathematics in automath”, in *Colloques Internationaux du Centre National de la Recherche Scientifique*, Section 10.2 introduces the notion of telescopes as generalized Σ -types, vol. 249, CNRS, 1975, pp. 135–145.